

Informatik
Hochschule Luzern (Schweiz)

**PRAKTISCHE DE-ANONYMISIERUNG IM
TOR-NETZWERK**
Einfluss von Circuit Padding auf Website-Fingerprinting im
Shadow Netzwerk Simulator

BACHELORARBEIT

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz) in Anbetracht
zur Erreichung des akademischen Grades *Bachelor in Informatik*.

von

Oliver Staub

in

Luzern (Schweiz)

Erklärung

Bachelorarbeit an der Hochschule Luzern - Informatik

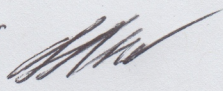
Titel:	Praktische De-Anonymisierung im Tor-Netzwerk
Studentin/Student:	Oliver Staub
Studiengang:	Bachelor in Informatik
Abschlussjahr:	2026
Betreuungsperson:	Dr. Radwan Eskhita
Expertenperson:	Enrico Senger
Auftraggeberin/Auftraggeber:	Hochschule Luzern

Codierung/Klassifizierung der Arbeit

- ☐ Öffentlich (Normalfall)
☒ Vertraulich

Eidesstattliche Erklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbständig und ohne unerlaubte fremde Hilfe angefertigt habe. Alle verwendeten Quellen, Literatur und Hilfsmittel (insbesondere künstliche Intelligenz oder sonstige verwendete Instrumente) wurden urheberrechts- und datenschutzkonform verwendet und wörtlich oder inhaltlich entnommene Stellen als solche kenntlich gemacht. Das Vertraulichkeitsinteresse des Auftraggebers wurde gewahrt und die Urheberrechtsbestimmungen der Hochschule Luzern respektiert.

Ort/Datum, Unterschrift Zürich, 12.06.2026 

Verdankung

Ich danke Dr. Radwan Eshkita für die Betreuung der Arbeit und Enrico Senger für die Übernahme der Expertenrolle. Ein besonderer Dank gilt den HSLU Lab Services für die freundliche und unkomplizierte Bereitstellung des Simulationsservers. Schliesslich danke ich meiner Familie, meinen Freundinnen und Freunden für die Unterstützung während des gesamten Studiums.

Oliver Staub, 2026

*Geistiges Eigentum gemäss der Studienordnung für die Ausbildung
an der Hochschule Luzern, FH Zentralschweiz*

Kurzfassung

Das Tor-Netzwerk anonymisiert Internetverbindungen. Mit Website Fingerprinting Angriffen können die besuchten Webseiten jedoch aus den Paketrichtungen und Zeitverläufen des verschlüsselten Verkehrs abgeleitet werden. Für den Deep Fingerprinting Klassifikator werden in kontrollierten Umgebungen Erkennungsraten von über 95 % berichtet. Als Gegenmassnahme fügt das in Tor integrierte Circuit Padding zusätzliche Zellen in den Verkehrsstrom ein, bisherige Simulationsstudien variierten diesen Padding-Modus jedoch nicht systematisch und nutzten eine proprietär abgeänderte Tor-Version.

Die vorliegende Arbeit schliesst diese Lücke und misst in einem kontrollierten Laborexperiment, wie sich die drei offiziellen Padding-Modi auf die Erkennungsrate eines Deep Fingerprinting Angriffs auswirken. Ergänzend wird das aggressivere Tamaraw Verfahren zum Vergleich herangezogen. Dazu wurde mit dem Shadow Simulator ein vollständiges, deterministisches Tor-Netzwerk im Massstab von einem Prozent (79 Relays) auf moderater Hardware und mit einer unmodifizierten Tor-Version aufgebaut. Statt direkter Tor Zellen werden PCAP-Dateien aufgezeichnet, was dem Modell einer passiven Beobachtung entspricht. Aus den aufgezeichneten Verkehrsmustern werden Richtungssequenzen extrahiert und mit der WFlib-Implementierung des Deep Fingerprinting Klassifikators ausgewertet.

Die Resultate zeigen ein gemischtes Bild, wonach das Padding nicht immer zu einer verringerten Erkennungsrate führt. Dies kann verschiedene Gründe haben, unter anderem die limitierte Grösse des simulierten Netzwerkes und die relativ homogenen Webseiten, welche getestet wurden. Tamaraw als alternative Verteidigungsmöglichkeit zeigte einen starken Effekt, allerdings zum Preis eines stark erhöhten Verkehrsaufkommens.

Als eigenständiger Beitrag senkt die aufgebaute, wiederverwendbare Werkzeugkette die Einstiegshürde für weitere Shadow-basierte Untersuchungen zu Website Fingerprinting und anderen Angriffen auf Tor.

Inhaltsverzeichnis

1	Problem, Fragestellung, Vision	1
1.1	Problemstellung	1
1.2	Fragestellung	1
1.3	Vision und erwartete Resultate	1
1.4	Aufgabenstellung	2
2	Stand der Technik	3
2.1	Tor	3
2.2	Angriffe auf Tor	7
2.3	Website-Fingerprinting	8
2.4	Shadow-Simulator	9
2.5	Website-Fingerprinting-Verteidigungen	12
3	Ideen und Konzepte	15
3.1	Bewertung des Stands der Forschung	15
3.2	Konzeptioneller Versuchsaufbau	15
3.3	Wahl des Simulators	15
3.4	Wahl des Klassifikators	15
3.5	Wahl des Szenarios	16
3.6	Wahl der Padding-Konfigurationen	16
3.7	Wahl der Tamaraw-Integration	16
3.8	Wahl des Hintergrundverkehrs	17
3.9	Wahl der Inhaltsquelle	17
3.10	Methodische Vereinfachung: Packet Capture (PCAP) statt Cell-Trace	17
3.11	Merkmalswahl: nur Richtungen	18
3.12	Zuordnung von Verkehr zu Webseiten	18
3.13	Circuit-Isolation per NEWNYM	18
3.14	Round-Robin-Schedule	18
3.15	Skalierung der Topologie	19
4	Methoden	20
4.1	Methodische Einordnung der Arbeit	20
4.2	Literaturrecherche	20
4.3	Experimentelles Vorgehen	21
4.4	Einsatz von KI-Werkzeugen	23
4.5	Projektplan	24
5	Realisierung	26
5.1	Übersicht des Versuchsaufbaus	26
5.2	Versuchsumgebung	27
5.3	Bereitstellung der Webinhalte	30

5.4	Monitor-Knoten und Verkehrserfassung	31
5.5	Aufbereitung der Daten für die Klassifikation	32
5.6	Training und Auswertung des Klassifikators	32
5.7	Tamaraw-Variante des Versuchsaufbaus	34
6	Validation und Evaluation	37
6.1	Validierung des Versuchsaufbaus	37
6.2	Klassifikationsresultate	37
6.3	Bandbreiten-Overhead und Kosten-Nutzen-Verhältnis	40
6.4	Ressourcen und Performance	42
6.5	Einordnung und Limitationen	43
6.6	Erreichung der Ziele und Forschungsfragen	45
7	Ausblick	47
7.1	Weiterentwicklung der Website-Fingerprinting-Experimente	47
7.2	Alternative Untersuchungsszenarien	49
7.3	Zusammenfassung der offenen Punkte	50
7.4	Reflexion	50
A	Aufgabenstellung	53
B	Serverumgebung	56
B.1	Bereitstellung	56
B.2	Hardware	56
B.3	Software	57
B.4	Datenablage und Verzeichnisstruktur	57
B.5	Ressourcenauslastung der Tamaraw-Läufe	58
C	Begleitende Materialien	60
C.1	Verzeichnisstruktur	60
C.2	Nicht enthaltene Daten	60
C.3	Reproduktion eines Resultats	61
C.4	Tabellarische Gesamtsicht aller Hauptexperimente	62
C.5	Verteilung der Sequenzlänge pro Besuch	64
D	Ausgewählte Konfigurations- und Code-Auszüge	65
D.1	Schedule-Datei	65
D.2	URL-Liste	65
D.3	NEWNYM-Skript	66

Abbildungsverzeichnis

2.1	Aufbau einer Tor-Zelle. Eigene Darstellung.	4
2.2	Vergleich von Transmission Control Protocol (TCP)-Segmenten und Tor-Zellen. Segmentgrenzen (gepunktet) fallen nicht mit Zellgrenzen zusammen: ein TCP-Segment kann mehrere ganze oder Teile von Zellen enthalten. Eigene Darstellung.	4
2.3	Onion Routing in Tor: Eine Nachricht wird vom Client mit den Schlüsseln aller drei Onion Router verschlüsselt. Jeder Hop entfernt genau eine Schicht, bis der Ausgangsknoten die Nachricht an das Ziel weiterleitet. Eigene Darstellung.	6
4.1	Projektplan der Bachelorarbeit mit Arbeitsphasen (blau) und fixen Meilensteinen.	25
5.1	Innerhalb der Simulation existiert ein vollständiges The Onion Router (Tor)-Netzwerk mit Eingangs-, Mittel- sowie Ausgangsknoten. Daneben laufen die für diese Arbeit eigens hinzugefügten Monitor-Clients (jeweils mit lokalem Tor-Daemon, gepatchtem <code>wget2</code> und PCAP-Aufzeichnung), ein Zim-Server für die Wikipedia-Inhalte sowie die von Tornettools generierten TGEN- und Markov-Hintergrund-Clients, deren Verkehr das Verhalten realer Tor-Nutzender nachbildet [1].	26
5.2	Konvertierungs-Pipeline: <code>pcap_to_npz.py</code> liest die Monitor-Pcaps und die <code>schedule.json</code> , schneidet pro Besuch die Paketrichtungen aus und schreibt sie zusammen mit den Klassen-Labels in eine kompakte <code>.npz</code> -Datei.	32
5.3	Tamaraw-Variante des Versuchsaufbaus. Gegenüber der Baseline-Topologie aus Abbildung 5.1 kommt der orange dargestellte Bridge-Knoten <code>wfbridge0</code> hinzu. Der monitor-seitige Tor-Daemon verbindet sich nicht direkt zu einem Guard, sondern über den Bridge-Knoten, dessen Relay-Prozess die geformten Zellen anschliessend regulär in das simulierte Tor-Netzwerk einspeist.	35
6.1	Accuracy des Deep Fingerprinting (DF)-Klassifikators für die vier untersuchten Verteidigungen. Die genauen Accuracy- und F1-Werte sind in Tabelle C.3 im Anhang aufgeführt.	37
6.2	Precision, Recall und F1-Wert im Open-World-Setup (80 überwachte Seiten in einem Topf von 280). Eigene Darstellung aus <code>Experiment-Logs.csv</code>	39
6.3	Konfusionsmatrizen im Closed-World-Setup mit 80 Klassen, normiert pro Zeile (Recall). Links die ungeschützte Baseline (OFF), Mitte mit aktivem Circuit Padding (ON), rechts mit Tamaraw. Eigene Darstellung.	40

6.4	Bandbreiten-Overhead (logarithmisch) gegen Accuracy-Reduktion bei 80 Klassen. Reduced und ON liegen im unteren linken Bereich (geringe Kosten, geringer Schutz), Tamaraw rechts oben (hoher Schutz, hohe Kosten). Eigene Darstellung.	42
6.5	Auslastung von CPU, Arbeitsspeicher und Disk auf shadowsrv-001 während der Tamaraw-Simulationen. Eigene Darstellung aus den perfmon - und dstat -Logs.	43

Tabellenverzeichnis

4.1	Im Rahmen der Arbeit eingesetzte KI-Werkzeuge.	24
5.1	Eigens entwickelte Pipeline-Skripte mit ihrer jeweiligen Aufgabe. . .	28
5.2	Drei untersuchte Padding-Konfigurationen.	29
5.3	Varianten der Wikipedia-ZIM-Datei (Stand 2026-02).	30
5.4	Hyperparameter des DF-Trainings.	33
6.1	Mittlerer Datenverkehr pro Besuch auf der Monitor-Schnittstelle und relativer Overhead gegenüber exp-baseline-80 . Eigene Mes- sung aus den Monitor-PCAPs mit <code>pcap_overhead.py</code>	41
B.1	Eckdaten der bereitgestellten virtuellen Maschine.	56
B.2	Hardware-Ausstattung der virtuellen Maschine.	56
B.3	Software-Komponenten auf der virtuellen Maschine.	57
B.4	Verzeichnisstruktur auf der VM.	57
B.5	exp-tamaraw-20v2 : CPU- und RAM-Auslastung der virtuellen Ma- schine zum jeweiligen Zeitpunkt seit Experimentstart.	58
B.6	exp-tamaraw-80 : CPU-, RAM- und Disk-Auslastung der virtuellen Maschine zum jeweiligen Zeitpunkt seit Experimentstart.	58
B.7	exp-tamaraw-openworld : CPU-, RAM- und Disk-Auslastung der virtuellen Maschine zum jeweiligen Zeitpunkt seit Experimentstart. Der Lauf wird bei 16.10 Stunden durch ENOSPC abgebrochen, als die Disk die volle Belegung von 156 GB erreicht.	59
C.1	Inhalt des beigelegten ZIP-Archivs.	60
C.2	Bewusst nicht ausgelieferte Bestandteile und ihre Reproduktion. . .	61
C.3	Aggregierte Kennzahlen der zwölf Hauptexperimente. Quelle: Ex- perimentLogs.csv	63
C.4	Verteilung der Sequenzlänge pro Besuch nach der Konvertierung mit <code>pcap_to_npz.py</code> . Eigene Auswertung über alle Samples des je- weiligen Datensatzes.	64

Glossar

Circuit Padding In Tor integriertes Verteidigungsframework, das zusätzliche Zellen in einen Circuit einfügt, um Verkehrsmuster zu verschleiern.. 1, 2, 6, 7, 12, 15, 16, 18, 20, 38, 39, 40, 41, 45, 46, 47, 48, 49

Closed-World Auswertungsszenario, in dem der Klassifikator nur Seiten aus einer bekannten, endlichen Menge überwachter Seiten unterscheiden muss.. 1, 8, 9, 11, 15, 16, 22, 23, 37, 38, 39, 40, 44, 45, 46, 47, 61, 64

Klasse Im Kontext dieser Arbeit eine zu unterscheidende Webseite. Closed-World mit 80 Klassen bedeutet 80 verschiedene Wikipedia-Artikel, die der Klassifikator unterscheiden muss.. 10, 44, 47

Monitor-Host Simulierter Tor-Client, der innerhalb der Shadow-Simulation Wikipedia-Seiten abrufen und dabei Verkehrsspuren für die Website-Fingerprinting-Klassifikation aufzeichnet.. 28, 64, 65

Open-World Auswertungsszenario mit einer überwachten Seitenmenge und einer offenen Sammelklasse für alle übrigen Seiten.. 8, 11, 15, 16, 22, 23, 38, 39, 43, 47, 64

Shadow Diskreter Netzwerkereignis-Simulator, der unmodifizierte Anwendungen wie den Tor-Daemon innerhalb einer deterministischen Simulation ausführt und in dieser Arbeit als Laborumgebung für die Website-Fingerprinting-Experimente dient.. 1, 9, 15, 26, 39, 45, 46, 47, 48, 49, 50, 64

Tamaraw Festrat-Padding-Verteidigung, die Pakete zu festen Zeitabständen sendet und damit Webseiten-spezifische Timing-Muster nivelliert.. 1, 12, 14, 16, 22, 23, 34, 35, 36, 37, 38, 39, 40, 41, 42, 45, 46, 48, 49, 58, 62, 64

ZIM Komprimiertes Container-Format für offline nutzbare Wikipedia-Snapshots, wird in dieser Arbeit von `zimsrv.py` ausgeliefert.. 27, 30, 47, 50, 60, 62

Akronyme

DF Deep Fingerprinting. vi, viii, 1, 2, 9, 15, 16, 23, 26, 27, 32, 33, 37, 38, 45, 61, 64

DOI Digital Object Identifier. 21

ML Machine Learning. 8

NSA National Security Agency. 7

PCAP Packet Capture. iv, vi, viii, 17, 18, 23, 26, 29, 32, 34, 40, 41, 43, 45, 48, 50, 57, 61, 65

PT Pluggable Transport. 16, 35, 36

SOCKS Socket Secure. 11, 27, 31, 57

SSH Secure Shell. 26, 27, 56

TCP Transmission Control Protocol. vi, 4, 40, 41, 45

Tor The Onion Router. vi, 1, 3, 5, 7, 8, 9, 26, 27, 28, 43, 44

WF Website Fingerprinting. 1, 2, 3, 7, 8, 9, 10, 12, 13, 15, 18, 22, 28, 44, 45, 46, 47, 49, 50

WTF-PAD Website Traffic Fingerprinting Protection with Adaptive Defense. 12, 13

1 Problem, Fragestellung, Vision

1.1 Problemstellung

Das Tor-Netzwerk anonymisiert Verbindungen, indem Pakete über mehrere verschlüsselte Relays geleitet werden. Trotz dieses Schutzes zeigen Website Fingerprinting (WF)-Angriffe, dass eine passive beobachtende Instanz zwischen Client und Eingangsknoten besuchte Webseiten allein anhand der Paketrichtungen und Zeitverläufe zuordnen kann. Mit dem DF-Klassifikator werden in einer Closed-World-Umgebung Erkennungsraten von über 95 % berichtet [2].

Als Gegenmassnahme hat das Tor-Netzwerk das Circuit Padding-Framework eingeführt, welches zusätzliche Zellen in den Verkehrsstrom einfügt¹.

Experimente zu Website-Fingerprinting lassen sich mit dem 2011 eingeführten Shadow-Simulator durchführen. Dieser ermöglicht es, ein vollständiges, deterministisches Tor-Netzwerk zu simulieren [3]. In der Forschung erwähnte Shadow-Simulationen stellen jedoch hohe Anforderungen an die Hardware und nutzen proprietär gepatchte Tor-Versionen zur Cell-Trace-Entnahme [4].

Die vorliegende Arbeit prüft, ob sich eine Simulation mit moderater Hardware und einer unmodifizierten Tor-Version durchführen lässt. Als Ziel soll der Effekt von Circuit Padding in der aufgebauten Simulationsumgebung beobachtet werden.

1.2 Fragestellung

Daraus ergeben sich die folgenden Forschungsfragen.²

1. Wie verändert sich die Erkennungsrate eines WF-Angriffs mit unterschiedlichen Circuit-Padding-Einstellungen?
2. Welche bekannten Limitationen simulierter und laborbasierter WF-Angriffe sind in der Forschungsliteratur dokumentiert, und wie sind die experimentellen Ergebnisse dieser Arbeit einzuordnen?
3. Welche zusätzliche Schutzwirkung gegen DF-Angriffe lässt sich mit dem neueren Tamaraw-Verfahren erzielen?

1.3 Vision und erwartete Resultate

Ziel der Arbeit ist es, den Einfluss von Circuit Padding auf WF-Angriffe unter kontrollierten Laborbedingungen empirisch zu evaluieren und die Ergebnisse in

¹<https://spec.torproject.org/padding-spec/circuit-level-padding.html>

²Die ursprüngliche Aufgabenstellung (Anhang A) listet zwei Forschungsfragen. Die dritte zur Tamaraw-Verteidigung wurde im Verlauf der Arbeit ergänzt.

den Stand der Forschung einzuordnen. Im Einzelnen werden folgende Resultate angestrebt.

- Eine aufgebaute Shadow-Simulationsumgebung in einer VM von labservices.ch.
- Ein in dieser Umgebung erstelltes WF-Datenset.
- Ein mit dem Tool WFlib durchgeführter DF-Angriff.

1.4 Aufgabenstellung

Die vollständige, von der auftraggebenden Stelle signierte, Aufgabenstellung findet sich in Anhang A. Sie umfasst den Aufbau der Shadow-Simulation und die Auswertung des Circuit Padding.

2 Stand der Technik

2.1 Tor

Tor ist ein verteiltes Overlay-Netzwerk mit niedriger Latenz, das TCP-basierte Anwendungen wie Web-Browsing, SSH oder Instant Messaging anonymisieren kann [5]. Das Netzwerk verspricht den Nutzenden Schutz vor Verfolgung, Zensur und Überwachung¹. Im Jahr 2018 ging man von bis zu acht Millionen täglichen Nutzern aus [6]. Diese hohe Verbreitung macht Tor zu einem attraktiven Ziel für unterschiedliche Klassen von Deanonymisierungsangriffen, wie Timing-Angriffe, Traffic-Correlation oder WF, die jeweils unterschiedliche Beobachtungspositionen und Annahmen voraussetzen.

Das aktuelle Tor-Design wurde 2004 von Dingledine, Mathewson und Syverson als zweite Generation des Onion-Routings vorgestellt und seither stetig weiterentwickelt [5].

Die folgenden Unterabschnitte beschreiben die für diese Arbeit relevanten Komponenten der Tor-Architektur sowie das Bedrohungsmodell, das dem Tor-Design zugrunde liegt. Die spezifischen Bedrohungsmodelle der einzelnen Angriffsklassen werden in den jeweiligen Abschnitten weiter unten behandelt.

2.1.1 Architektur Tor

Das Tor-Netzwerk gliedert sich in mehrere Rollen. Der Client ist der Rechner der nutzenden Person. Auf ihm läuft die Tor-Software, die Anwendungs-Verbindungen entgegennimmt, einen Pfad durch das Netzwerk wählt und die Daten schichtweise verschlüsselt [5].

Die Onion Router (kurz: Relais) sind freiwillig betriebene Server, welche diese verschlüsselten Daten weiterleiten. Pro Verbindung kommen drei Relais zum Einsatz, in den Rollen Eingangs-, Mittel- und Ausgangsknoten [7]. Damit ein Client überhaupt weiss, welche Relais derzeit verfügbar sind, betreiben wenige vertrauenswürdige Stellen die Rolle der Verzeichnisserver (Directory Authorities). Sie sammeln in regelmässigen Abständen die Eigenschaften aller Relais (IP, Bandbreite, Flags), stimmen über diese Liste ab und veröffentlichen ein gemeinsames, signiertes Verzeichnisdokument, das ein Client vor dem Aufbau einer Verbindung herunterlädt [8].

2.1.2 Tor Zellen

Die kleinste Übertragungseinheit ist eine Zelle fester Grösse, im ursprünglichen Design 512 Bytes. Unterschieden wird zwischen Control Cells (z. B. create, destroy,

¹<https://www.torproject.org/de/> (Zugriff: 2026-04-08).

padding) und Relay Cells, die Ende-zu-Ende-Stream-Daten transportieren [5].

Jede Zelle besteht aus einem kurzen Header und einem Payload-Bereich. Der Header enthält eine Circuit-ID, die den Schaltkreis innerhalb einer Hop-Verbindung identifiziert, sowie ein Befehlsbyte (Command), das den Zelltyp angibt [5]. Bei Relay Cells ist der Payload zusätzlich strukturiert und enthält einen inneren Relay-Header mit Stream-ID, einer Längenangabe und einer Integritätsprüfsumme (Digest) sowie das eigentliche Datenfeld, in dem die Anwendungs-Nutzdaten transportiert werden [5]. Abbildung 2.1 fasst diesen Aufbau zusammen.

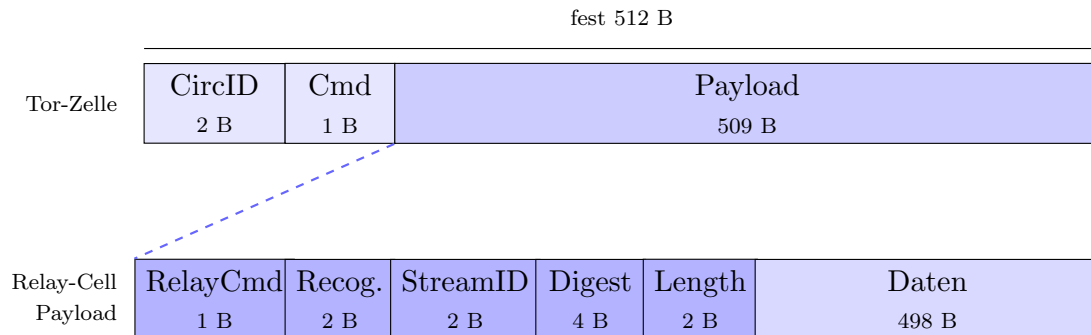


Abbildung 2.1: Aufbau einer Tor-Zelle. Eigene Darstellung.

Im Gegensatz zu TCP-Segmenten, deren Nutzdaten-Grösse durch die Maximum Transmission Unit nach oben begrenzt ist, besitzt jede Tor-Zelle dieselbe feste Länge. Zwischen jedem Paar aufeinanderfolgender Knoten eines Circuits, also auch zwischen Client und Eingangsknoten, werden Zellen über eine TLS-gesicherte TCP-Verbindung transportiert, sodass ein einzelnes TCP-Segment je nach Auslastung eine oder mehrere vollständige Zellen tragen kann [5]. Abbildung 2.2 stellt den Unterschied schematisch dar: TCP-Segmente besitzen variable Länge und folgen der Anwendungslogik, während Tor-Zellen eine einheitliche Länge aufweisen und unabhängig von den Segmentgrenzen im Bytestrom liegen.

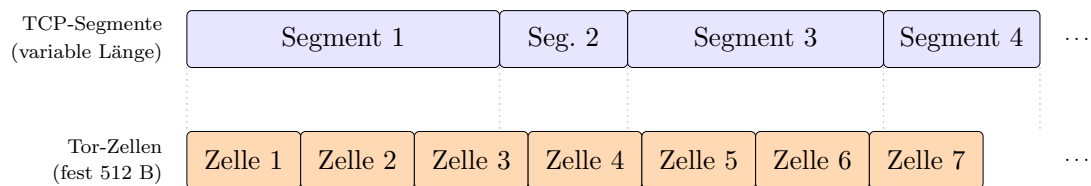


Abbildung 2.2: Vergleich von TCP-Segmenten und Tor-Zellen. Segmentgrenzen (gepunktet) fallen nicht mit Zellgrenzen zusammen: ein TCP-Segment kann mehrere ganze oder Teile von Zellen enthalten. Eigene Darstellung.

Die feste Zellgrösse soll Verkehrsmuster auf Paketebene vereinheitlichen. Sichtbar bleiben für eine beobachtende Instanz zwischen Client und Eingangsknoten jedoch Anzahl und Richtung der Zellen, das heisst, ob eine Zelle vom Client zum

Eingangsknoten (ausgehend) oder vom Eingangsknoten zum Client (eingehend) übertragen wird. Genau diese Sequenz aus eingehenden und ausgehenden Zellen bildet das Eingangssignal späterer Website-Fingerprinting-Klassifikatoren [4].

2.1.3 Circuits und Streams

Ein Circuit ist ein kryptografisch geschützter Pfad durch das Netzwerk und enthält in der Regel drei Knoten: Eingangs-, Mittel- und Ausgangsknoten. Der Eingangsknoten kennt den Client und den Mittelknoten, nicht aber den Ausgangsknoten oder das Ziel. Der Mittelknoten kennt nur seine beiden Nachbarn. Der Ausgangsknoten kennt das Ziel, nicht aber den Client.

Der Client baut den Circuit schrittweise auf: Mit dem ersten Hop wird per Diffie-Hellman ein Sitzungsschlüssel ausgehandelt, anschliessend wird der Circuit über bereits etablierte Hops um jeweils einen weiteren Onion Router erweitert [5].

Über einen einzelnen Circuit können mehrere TCP-Streams parallel geleitet werden, was die Anzahl rechenintensiver Public-Key-Operationen reduziert. Um die Privatsphäre zu erhöhen, rotiert der Tor-Client Circuits periodisch und nutzt für neu eintreffende Streams den jeweils neuesten offenen Circuit [5]. Im Tor Browser werden zusätzlich alle Streams derselben First-Party-Domain (der in der Adresszeile angezeigten Domain) auf denselben Circuit gebündelt [4].

2.1.4 Onion Routing

Das namensgebende Verfahren von Tor (The Onion Router) ist das Onion Routing. Der Client verschlüsselt eine Nachricht schichtweise mit den Sitzungsschlüsseln aller drei Onion Router des Circuits, sodass jeder Hop genau eine Schicht entfernt und nur seine direkten Nachbarn kennt [5]. Dadurch besitzt kein einzelner Onion Router gleichzeitig Kenntnis über Quelle und Ziel der Kommunikation [5].

Abbildung 2.3 veranschaulicht diesen Vorgang: Vom Client zum Ziel reduziert sich die Anzahl der Verschlüsselungsschichten an jedem Hop um eine, bis der Ausgangsknoten die ursprüngliche Nachricht ohne Tor-Verschlüsselung an das Ziel weiterleitet.

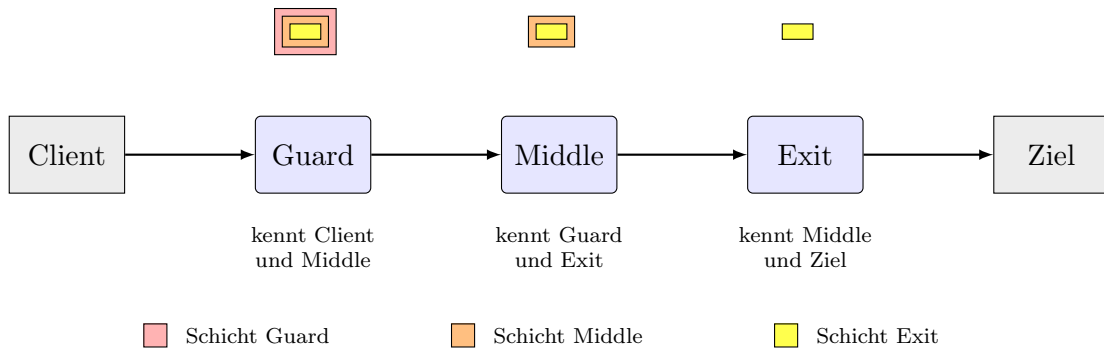


Abbildung 2.3: Onion Routing in Tor: Eine Nachricht wird vom Client mit den Schlüsseln aller drei Onion Router verschlüsselt. Jeder Hop entfernt genau eine Schicht, bis der Ausgangsknoten die Nachricht an das Ziel weiterleitet. Eigene Darstellung.

2.1.5 Bedrohungsmodell

Tor ist explizit nicht gegen eine global agierende passive Beobachtung ausgelegt. Angenommen wird stattdessen eine angreifende Partei, die einen Teil des Netzverkehrs beobachten, den Verkehr modifizieren oder verzögern, eigene Onion Router betreiben sowie einen Teil bestehender Router kompromittieren kann [5]. Das Designziel ist die Abwehr von Traffic-Analysis-Angriffen, nicht aber von Traffic-Confirmation-Angriffen, bei denen beide Enden einer Verbindung beobachtet werden [5].

2.1.6 Kein Padding im Originaldesign

Im ursprünglichen Tor-Design war kein Padding oder Traffic Shaping vorgesehen. Begründet wird dies mit den Kosten solcher Verfahren und damit, dass zum damaligen Zeitpunkt kein praktikables, nachweislich anonymitätssteigerndes Padding-Schema existierte [5]. Genau an dieser Lücke setzen spätere Arbeiten zum Circuit Padding an, das in den folgenden Abschnitten behandelt wird.

2.1.7 Tor Browser

Das Tor-Projekt stellt mit dem Tor Browser eine modifizierte Variante von Firefox bereit, die für die Nutzung mit Tor optimiert ist und zusätzlich Browser-Fingerprinting erschwert². Der Browser greift via SOCKS auf den lokalen Tor-Daemon zu, welcher den Internetverkehr in Form von Tor-Zellen über das Tor-Netzwerk weiterleitet [4].

²<https://www.torproject.org/de/> (Zugriff: 2026-04-08).

2.2 Angriffe auf Tor

Das in Abschnitt 2.1.5 skizzierte Bedrohungsmodell von Tor schliesst eine global agierende passive Beobachtung explizit aus und richtet sich gegen eine angreifende Partei mit einer Teilsicht auf das Netz [5]. Innerhalb dieses Modells lassen sich verschiedene Deanonymisierungsangriffe ausmachen.

2.2.1 Angriffsklassen im Überblick

Bei einem **Website-Fingerprinting-Angriff** beobachtet die angreifende Partei ausschliesslich die Verbindung zwischen Client und Eingangsknoten und kennt das Kommunikationsziel nicht. Aus dem Muster der Tor-Zellen wird abgeschätzt, welche Webseite aus einer der angreifenden Partei im Voraus bekannten Menge aufgerufen wurde [4]. Diese Angriffsklasse steht im Mittelpunkt der vorliegenden Arbeit und wird in Abschnitt 2.3 ausführlich behandelt.

Bei einem **Traffic-Correlation-Angriff** beobachtet die angreifende Partei hingegen beide Enden eines anonymisierten Kommunikationspfads gleichzeitig. Aus den beiden Verkehrsspuren werden zueinander gehörende Flow-Paare identifiziert, womit eine direkte Verbindung zwischen Absender und Ziel hergestellt wird [9]. Da die Aussage nicht über eine bekannte Inhaltsmenge, sondern über eine Identitätszuordnung getroffen wird, lassen sich auf diesem Weg beliebige Ziele deanonymisieren, ohne dass diese im Voraus bekannt sein müssen.

Eine dritte Klasse bilden **Seitenkanal- und Latenzangriffe**. Anstatt den Tor-Verkehr direkt aufzuzeichnen, leitet ein solcher Angriff Information aus indirekt beobachtbaren Grössen wie Reaktionszeiten, Auslastungsschwankungen einzelner Relais oder Pfad-Entscheidungen ab und benötigt damit eine schwächere Beobachtungsposition als die beiden zuvor genannten Klassen. Murdoch und Danezis zeigten bereits 2005, dass ein einzelner kompromittierter Tor-Knoten zusammen mit aktiven Latenzproben genügen kann, um die Auslastung anderer Relais zu schätzen und so wahrscheinliche Pfade zu rekonstruieren [10].

Im Rahmen dieser Arbeit wird ausschliesslich WF experimentell untersucht. Die übrigen Klassen werden hier nur abgegrenzt, da sie für die Einordnung der Schutzwirkung von Circuit Padding wichtig sind: Padding fügt Schein-Verkehr ausschliesslich auf den Tor-internen Verbindungen ein und adressiert damit primär den einseitig beobachtenden WF-Angriff, nicht jedoch eine angreifende Partei, die beide Endpunkte beobachtet oder Seitenkanäle ausnutzt.

2.2.2 Angriffe in der Praxis

Die durch Edward Snowden veröffentlichten Folien der National Security Agency (NSA) und des GCHQ, insbesondere *Tor Stinks* aus dem Jahr 2012, nennen als operative Methoden gegen Tor das **Cookie-Tracking** auf Exit-Verkehr, das Markieren von User-Agents sowie das Betreiben **eigener Exit-Knoten**, jedoch keine WF-basierte Inhalts-Klassifikation [11]. Konkret nutzt das Cookie-Tracking aus,

dass persistente Web-Cookies, etwa von Werbenetzwerken wie DoubleClick oder Google Analytics, die Tor-Anonymisierung auf der Anwendungsschicht überleben. Wird derselbe Browser oder dasselbe Browser-Profil für Tor- und Nicht-Tor-Verkehr verwendet, lässt sich derselbe Cookie-Wert sowohl im beobachteten Exit-Traffic als auch im Direkt-Verkehr mit bekannter IP-Adresse wiederfinden, womit eine Tor-Sitzung an eine reale Identität rückgebunden wird [11]. Genau diesem Muster begegnet der Tor Browser durch eine konsequente Isolation der Cookies pro First-Party-Domain und durch das automatische Verwerfen von Browser-Zuständen zwischen Sitzungen.

Browser-Exploits nutzen Schwachstellen im Tor Browser aus, um nach erfolgreicher Code-Ausführung identifizierende Informationen wie die reale IP-Adresse direkt am Tor-Kanal vorbei zu exfiltrieren, wie beim Browser-Exploit gegen Freedom Hosting 2013 [12] oder beim FBI-Einsatz gegen den Hidden Service Playpen [13].

Sybil-Operationen bestehen darin, eine grosse Zahl bössartiger Relais unter einer Identität zu betreiben, um durch gleichzeitige Kontrolle mehrerer Positionen eines Schaltkreises Traffic-Confirmation zu ermöglichen, wie der Relay-Early-Angriff von 2014 [14] und die seit 2017 dokumentierte KAX17-Operation mit über 900 Relais [15] zeigen.

2.3 Website-Fingerprinting

WF-Angriffe können herausfinden, welche Webseite über Tor aufgerufen wurde, trotz Tor-Verschlüsselung. Die angreifende Partei zeichnet dazu den Datenverkehr zwischen Client und Eingangsknoten auf, entweder am Eingangsknoten selbst [4] oder als passive lokale beobachtende Instanz [16]. Aus Anzahl und Richtung der Tor-Zellen, optional ergänzt um Zeitstempel, leitet ein Machine Learning (ML)-Klassifikator ein Verkehrsmuster pro Webseite ab.

Die angreifende Partei definiert eine Menge überwachter Seiten, alle übrigen gelten als nicht-überwacht. Der Klassifikator wird zweistufig trainiert: Zuerst entscheidet er binär, ob eine beobachtete Zellsequenz zu einer überwachten Seite gehört, anschliessend bestimmt eine Multiclass-Klassifikation die konkrete Seite [4]. Pro beobachteter Sequenz ergeben sich die vier üblichen Ausgänge: True Positive, True Negative, False Positive und False Negative. Welche Metriken daraus gebildet werden, wird im Abschnitt 2.4.5 (Closed-World und Open-World) behandelt.

2.3.1 Website-Fingerprinting in der Praxis

Eine für die vorliegende Arbeit zentrale Einordnung ergibt sich aus einem Blick auf öffentlich bekannte Tor-Deanonymisierungen. Trotz mehr als fünfzehn Jahren intensiver WF-Forschung existiert kein öffentlich dokumentierter Fall, in dem die im akademischen Sinn definierte WF-Klassifikation eingesetzt wurde, um eine reale Person zu deanonymisieren.

Mehrere Faktoren erklären diese Lücke zwischen Labor und Praxis. Wang und

Goldberg argumentieren, dass die in akademischen Closed-World-Experimenten ausgewiesenen Trefferquoten unter realistischen Bedingungen rasch in unbrauchbar viele False Positives umschlagen [16]. Cherubin et al. bestätigen diese Lücke 2022 mit einer Evaluation auf echtem Tor-Exit-Verkehr, bei der die Accuracy bereits ab 25 überwachten Seiten unter 80 % fiel [17].

Hinzu kommt, dass für staatliche angreifende Parteien einfacher zu betreibende Alternativen zur Verfügung stehen, die mit höherer Trefferwahrscheinlichkeit zu einer eindeutigen Identifikation führen (vgl. Abschnitt 2.2.2).

WF-ähnliche Verfahren werden in der Praxis dennoch operativ eingesetzt, jedoch zu einem anderen Zweck. Zensurregimes wie die Great Firewall of China nutzen Protokoll-Fingerprinting, um Tor-Verbindungen am verwendeten TLS-Handshake zu erkennen und zu blockieren, also an der Auswahl und Reihenfolge der angebotenen Ciphersuites sowie an weiteren Verbindungsparametern, die sich von gewöhnlichem HTTPS-Verkehr unterscheiden [18]. Diese Erkennung zielt auf die Tatsache der Tor-Nutzung selbst und nicht auf den Inhalt der besuchten Webseiten und stellt damit ein anderes Bedrohungsmodell dar, als das in der vorliegenden Arbeit untersucht.

Die experimentelle Auseinandersetzung mit WF-Verteidigungen behält dennoch ihre Berechtigung. Sie evaluiert eine präventive Massnahme gegen ein dokumentiert mögliches, in der akademischen Literatur kontinuierlich verfeinertes Angriffsmuster, dessen praktische Anwendung zwar bisher ausgeblieben ist aber denkbar bleibt.

2.4 Shadow-Simulator

2.4.1 Shadow-Simulator

Shadow ist ein Ereignissimulator, der unmodifizierte Netzwerkanwendungen wie den realen Tor-Daemon, den Verkehrsgenerator TGen oder auch wget2 innerhalb einer Simulation ausführen kann [19].

Shadow fängt netzwerk- und zeitbezogene Aufrufe ab und beantwortet sie aus dem simulierten Netzwerk heraus, während alle übrigen Syscalls nativ ausgeführt werden [3]. Da die Simulationszeit explizit verwaltet und von der Wanduhrzeit entkoppelt ist, skaliert Shadow unabhängig von der Hardware-Geschwindigkeit, wobei die Laufzeit grosser Simulationen je nach Versuch von wenigen Stunden bis zu mehreren Wochen reichen kann [1].

Jansen und Wails zeigen, dass ein vollständig in Shadow trainierter DF-Klassifikator auf Live-Tor-Daten 82 % der Aufrufe korrekt klassifiziert [4]. Im Live-Tor-Netzwerk verändern unkontrollierte Faktoren wie laufende Denial-of-Service-Attacken oder überlastete Relais die Verkehrsmuster und erschweren so die Reproduzierbarkeit, was in der deterministischen Shadow-Umgebung nicht auftritt [4]. Shadow eignet sich damit nicht nur für isolierte Vergleichsstudien, sondern auch für das Training produktiv einsetzbarer WF-Klassifikatoren.

2.4.2 Tornettools

Tornettools ist der Modellierungsbaustein, der Shadow für realistische Tor-Experimente konfiguriert, und wurde 2021 von Jansen, Tracey und Goldberg veröffentlicht [1]. Aus Tor-Konsensdokumenten und Server-Deskriptoren des CollecTor-Archivs [20] sowie aus PrivCount-Verkehrsmessungen an Live-Tor-Exit-Relais [6] erzeugt es in einer zweistufigen Pipeline (rechenintensive Staging-Phase, rechengünstige Generation-Phase) eine skalierbare Shadow-Konfiguration [1]. Den Anwendungs-Verkehr der Hintergrund-Clients modelliert der Generator TGen über drei Markov-Modelle, deren Parameter aus PrivCount stammen und damit Volumen sowie zeitliche Struktur realer Tor-Nutzung nachbilden [1].

Die konkrete Werkzeugkette und die in dieser Arbeit gewählten Parameter werden im Abschnitt 5.2 beschrieben.

2.4.3 Validierung der Modellierung

Jansen et al. verglichen ihre Simulationen bei verschiedenen Netzwerkskalen mit Live-Tor-Metrics, wobei die Verteilungen von Circuit Build Time, Round-Trip-Time und Time-to-Last-Byte weitgehend mit den realen Messwerten übereinstimmten [1]. Eine spätere Cross-Dataset-Validierung zeigte zudem, dass ein ausschliesslich auf Shadow-Daten trainierter CUMUL-Klassifikator auf Live-Tor-Traces eine Accuracy von 86 % erreichte, was die Eignung der Modellierung speziell für WF-Studien bestätigt [4].

2.4.4 Website-Fingerprinting-Klassifikatoren

In der WF-Forschungsliteratur haben sich vier Klassifikatoren als Vergleichsbasis etabliert, die sich entlang zweier Dimensionen unterscheiden: klassische Machine-Learning-Verfahren gegenüber neuronalen Netzen sowie zeitbasierte gegenüber zeitunabhängigen Ansätzen [4].

CUMUL [21] verwendet eine Support Vector Machine über kumulierten Zellrichtungssequenzen und arbeitet zeitunabhängig. k-Fingerprinting [22] kombiniert einen Random Forest mit k-Nearest-Neighbors auf Basis von Summary-Statistiken der Zellsequenz, wobei Timing-Features (Inter-Arrival-Zeiten) genutzt werden. Deep Fingerprinting [2] ist ein zeitunabhängiges Convolutional Neural Network, das direkt auf Zellrichtungssequenzen arbeitet und in früheren Vergleichen sowohl CUMUL als auch k-FP übertroffen hat. Tik-Tok [23] verwendet die DF-Architektur, ergänzt die Eingabe aber um Zeitstempel und reagiert dadurch empfindlicher auf variierende Netzwerkbedingungen [4].

2.4.5 Closed-World- und Open-World-Szenarien

Vorab werden die in dieser Arbeit verwendeten Begriffe abgegrenzt. *Webseite* bezeichnet das im Datensatz enthaltene Untersuchungsobjekt, also eine konkrete URL beziehungsweise einen Wikipedia-Artikel. *Klasse* bezeichnet das vom Klas-

sifikator unterschiedene Label. Im Closed-World-Szenario fällt beides zusammen, im Open-World-Szenario nicht.

In der Website-Fingerprinting-Forschungsliteratur werden zwei grundlegende Evaluationsszenarien unterschieden: das Closed-World- und das Open-World-Szenario [16, 4].

Im Closed-World-Szenario wird angenommen, dass die nutzende Person ausschliesslich Webseiten aus einer der angreifenden Partei bekannten, endlichen Liste besucht. Der Klassifikator muss lediglich entscheiden, welche der bekannten Seiten besucht wurde. Da alle Klassen gleich häufig im Testdatensatz vertreten sind, wird die Klassifikationsgenauigkeit (Accuracy) als Performanzmass verwendet. Dieses Szenario eignet sich gut für kontrollierte Experimente, ist jedoch unrealistisch, da es die Existenz unbekannter Webseiten vollständig ausschliesst [16].

Im Open-World-Szenario hingegen definiert die angreifende Partei eine Menge sensibler Seiten W_α , die erkannt werden sollen, während die nutzende Person beliebige weitere Seiten besuchen kann – sowohl der angreifenden Partei bekannte, nicht-sensitive Seiten W_β als auch völlig unbekannte Seiten $W_\emptyset$. In der Umsetzung werden die K überwachten Seiten als eigene Klassen geführt und alle übrigen in einer Sammelklasse zusammengefasst, sodass effektiv $K + 1$ Klassen unterschieden werden. Der Klassifikator muss somit zunächst eine binäre Entscheidung treffen: Gehört eine beobachtete Zellsequenz zu einer sensiblen Seite oder nicht? Da die Klassen in diesem Szenario stark unbalanciert sind – die Anzahl nicht-sensibler Seiten übersteigt die der sensiblen um ein Vielfaches – ist Accuracy kein geeignetes Mass. Stattdessen werden die True Positive Rate (TPR) und die False Positive Rate (FPR) herangezogen. Eine niedrige FPR ist dabei besonders wichtig, da bei der geringen Basisrate sensibler Seitenbesuche selbst eine scheinbar kleine FPR zu einer grossen Anzahl falsch-positiver Alarme führen kann [16].

Wang und Goldberg betonen, dass frühere WF-Arbeiten häufig unter vereinfachten Closed-World-Annahmen evaluiert wurden, was die tatsächliche Bedrohung durch WF-Angriffe nur unzureichend widerspiegelt [16]. Jansen und Wails haben jedoch gezeigt, dass die Closed-World-Accuracy selbst bei variierenden Netzwerkbedingungen stabil bleibt (Schwankungen von nur 1–3 %), während TPR und FPR im Open-World-Szenario deutlich empfindlicher auf veränderte Bedingungen reagieren [4].

2.4.6 Webclient-Anpassung wget2

Da Tor seinen Dienst über einen Socket Secure (SOCKS)-Proxy bereitstellt, Webcrawler wie wget2 jedoch standardmässig nur HTTP-Proxies unterstützen, erweiterten Jansen und Wails wget2 um einen SOCKS-Client. Erst diese Modifikation ermöglichte es, wget2-Anfragen über den lokalen Tor-Prozess zu leiten und Zellsequenzen am Entry Relay aufzuzeichnen. Darüber hinaus wurde wget2 so konfiguriert, dass es alle eingebetteten Ressourcen einer Seite parallel herunterlädt und denselben User-Agent wie der Tor Browser verwendet. Durch lokale Tests wurde verifiziert, dass wget2 und der Tor Browser einen identischen Satz an Ressourcen

vom Server abrufen [4]. Die gepatchte wget2-Version ist im zugehörigen Artefakt-Repository öffentlich verfügbar.

2.5 Website-Fingerprinting-Verteidigungen

In den vorigen Abschnitten wurde WF als Angriffsklasse behandelt. Der folgende Abschnitt schliesst diese Vorbetrachtung mit den möglichen Verteidigungen WTF-PAD als ressourcenschonende Vorlage für das in Tor real ausgerollte Circuit Padding, und Tamaraw als alternatives, aufwendigeres Verfahren.

2.5.1 Trade-off zwischen Sicherheit und Overhead

Eine WF-Verteidigung verändert die für den Klassifikator charakteristischen Merkmale einer Zellsequenz, also Reihenfolge, Volumen und Inter-Paket-Zeiten, so dass unterschiedliche Webseiten weniger unterscheidbar werden [24]. Zwei Hebel stehen dafür zur Verfügung. Erstens das Einfügen zusätzlicher Dummy-Zellen (Padding), was die belegte Bandbreite erhöht. Zweitens das Verzögern realer Zellen (Delay), was die Latenz erhöht.

Für Tor, dessen Designziel eine niedrige Latenz für interaktive Anwendungen vorsieht, scheidet der zweite Hebel weitgehend aus [25]. Cai et al. zeigen zudem für reine Bandbreiten-Verteidigungen eine prinzipielle untere Schranke: Jede Verteidigung, die ein gegebenes Sicherheitsniveau erreicht, muss mindestens eine bestimmte Bandbreitenmenge investieren [24]. Konkrete Verteidigungen lassen sich anhand dieser Schranke entlang eines Spektrums einordnen, das von hoher Sicherheit bei hohem Overhead bis zu moderater Sicherheit bei geringem Overhead reicht.

2.5.2 WTF-PAD

Website Traffic Fingerprinting Protection with Adaptive Defense (WTF-PAD) wurde 2015 von Juarez, Imani, Perry, Diaz und Wright als effizientes Padding-Verfahren für Tor vorgeschlagen [26] und bildet die konzeptionelle Grundlage des später in Tor integrierten Adaptive-Padding-Frameworks [25].

Das Ziel von WTF-PAD ist es, die für WF-Klassifikatoren auffälligen Pausen zwischen Zellbursts zu maskieren, indem genau dort Dummy-Zellen eingestreut werden [26]. Dafür beobachtet eine pro Endpunkt laufende Komponente den ausgehenden Zellstrom und unterscheidet zwei Phasen, die als Burst-Modus und Gap-Modus bezeichnet werden.

Solange reale Zellen versendet werden, bleibt der Mechanismus im Burst-Modus und leitet jede Zelle ohne Verzögerung weiter. Nach jeder versendeten realen Zelle startet WTF-PAD jedoch eine kurze zufällige Wartezeit. Trifft innerhalb dieser Wartezeit eine weitere reale Zelle ein, wird sie sofort versendet und es startet eine neue Wartezeit. Trifft hingegen keine reale Zelle ein, gilt der aktuelle Burst als beendet.

Genau in diesem Moment wechselt WTF-PAD in den Gap-Modus und sendet selbst Dummy-Zellen. Deren zeitliche Abstände stammen aus einem Histogramm, welches echte Burst-Verläufe nachbildet. So entsteht im sonst leeren Gap-Intervall ein künstlicher Pseudo-Burst, der für eine aussenstehende beobachtende Instanz wie weitere echte Aktivität aussieht. Sobald eine reale Zelle eintrifft, kehrt der Mechanismus in den Burst-Modus zurück und der Zyklus beginnt von vorne [26]. Reale Zellen werden zu keinem Zeitpunkt aufgehalten, weshalb für die Anwendung kein Latenz-Overhead entsteht.

In einer geschlossenen Auswertung gegen die seinerzeit stärkste angreifende Partei reduziert WTF-PAD die Trefferquote von rund 91 % auf etwa 20 % bei einem Bandbreiten-Overhead von weniger als 60 % [26]. Damit liegt das Verfahren deutlich näher am Anforderungsprofil von Tor als die BuFLO-Familie und stellt die direkte Vorlage für das im folgenden Abschnitt beschriebene Circuit-Padding-System dar.

2.5.3 Tor Circuit Padding

Adaptive-Padding-Framework

Einen wesentlichen Anlass für die Einführung von Padding in Tor lieferte 2015 ein *Circuit-Fingerprinting*-Angriff von Kwon et al. [27]. Sie zeigten, dass eine beobachtende Instanz am Eingangsknoten allein anhand der Zellsequenz des Circuit-Aufbaus die verschiedenen Circuit-Typen unterscheiden kann, also einen gewöhnlichen Browsing-Circuit von einem Introduction- oder Rendezvous-Circuit eines Onion Services. Damit liess sich erkennen, dass und wann eine Person einen Onion Service ansteuert oder betreibt, was die Anonymitätsmenge der Nutzenden und Betreibenden von Onion Services erheblich schwächte. Die ersten produktiv ausgerollten Padding-Maschinen adressieren genau dieses Muster, indem sie die Aufbau-Sequenzen von Onion-Service-Circuits durch eingestreute Dummy-Zellen den Mustern gewöhnlicher Verzeichnisabfragen angleichen [25].

Anstatt diesen Angriff mit einer Punktlösung zu beheben, entschied sich das Tor-Projekt für ein generisches Padding-Framework, das seit der Tor-Version 0.4.0.1-alpha fester Bestandteil des Tor-Daemons ist und über einen eigenen Padding-Negotiation-Mechanismus zwischen Client und Relais aktiviert wird [25]. Die zugrunde liegende Adaptive-Padding-Idee entspricht dem im vorigen Abschnitt beschriebenen WTF-PAD-Mechanismus, das Framework wurde aber bewusst als Plattform konzipiert, auf der künftig auch WF-Verteidigungen ohne weitere Eingriffe in den Tor-Daemon lauffähig sein sollen. Auch wenn die aktuell ausgerollten Maschinen primär gegen Circuit Fingerprinting gerichtet sind, verändern sie durch die zusätzlich eingestreuten Zellen die für WF-Klassifikatoren sichtbaren Verkehrsmuster, was Gegenstand der experimentellen Auswertung dieser Arbeit ist.

Padding-Modi off, on und reduced

Die in dieser Arbeit verglichenen Padding-Konfigurationen werden über zwei Optionen in der torrc-Datei der simulierten Tor-Clients gesteuert. Diese drei Konfi-

gurationen bilden die unabhängige Variable des experimentellen Vergleichs.

Der Modus `off` entspricht der Einstellung `CircuitPadding 0` und deaktiviert sämtliche Padding-Maschinen. Er bildet die Baseline ohne Verteidigung ab und entspricht in der Wirkung dem ursprünglichen Tor-Design von Dingledine et al., das explizit auf Padding verzichtete [5].

Der Modus `on` aktiviert mit `CircuitPadding 1` die im Tor-Konsens aktuell deployten Standard-Maschinen.

Der Modus `reduced` kombiniert `CircuitPadding 1` mit der zusätzlichen Option `ReducedCircuitPadding 1`. Diese Option ist für mobile oder bandbreitenschwache Clients vorgesehen und reduziert Anzahl und Intensität der aktiven Padding-Maschinen, sodass der Bandbreiten-Overhead sinkt.

2.5.4 Tamaraw

Tamaraw wurde 2014 von Cai et al. vorgeschlagen und verfolgt das Ziel, den Verkehr unterschiedlicher Webseiten so weit zu vereinheitlichen, dass Seiten ähnlicher Grösse für einen Klassifikator nicht mehr unterscheidbar sind [24].

Im Kern sendet Tamaraw Pakete in einem festen Takt, unabhängig davon, ob gerade reale Daten zu übertragen sind. Pakete besitzen eine einheitliche Grösse von 750 Byte und werden pro Richtung in konstanten Zeitabständen versendet, wobei die Takte für eingehende und ausgehende Richtung unabhängig voneinander gewählt werden. Steht in einem Sende-Slot eine reale Zelle bereit, wird sie versendet, andernfalls wird der Slot durch eine Dummy-Zelle aufgefüllt. Reale Daten, die zwischen zwei Slots eintreffen, müssen bis zum nächsten Slot warten, wodurch eine systematische Verzögerung entsteht. Am Ende einer Übertragung wird die Gesamtzahl der versendeten Pakete pro Richtung auf das nächste Vielfache eines Parameters aufgefüllt, sodass Seiten ähnlicher Grösse in identischen Anonymitäts-Buckets landen [24].

Die systematische Verzögerung realer Zellen macht das Verfahren jedoch für Tor ungeeignet und es wurde nicht eingeführt [25].

3 Ideen und Konzepte

Dieses Kapitel begründet, wie die in Kapitel 2 beschriebenen Bausteine Tor, Website-Fingerprinting, Circuit Padding und Shadow zu einem Versuchsaufbau zusammengeführt werden. Die technische Umsetzung folgt in Kapitel 5.

3.1 Bewertung des Stands der Forschung

Padding und Website-Fingerprinting wurden in der Forschungsliteratur untersucht, aber nicht in einer Shadow-Simulation systematisch gekoppelt. Jansen und Wails messen DF in Shadow, variieren aber den Padding-Modus nicht [4]. Sie arbeiten zudem mit einem zellgenauen Cell-Trace aus einer gepatchten Tor-Version, während die vorliegende Arbeit am Monitor mitgeschnittene TCP-Pakete verwendet. Die vorliegende Arbeit füllt diese Lücke, indem sie die drei in Tor angebotenen Padding-Konfigurationen unter sonst konstanten Bedingungen gegen DF misst.

3.2 Konzeptioneller Versuchsaufbau

Der Versuchsaufbau folgt einem kontrollierten Laborexperiment, in dem der Padding-Modus gezielt variiert und alle übrigen Parameter konstant gehalten werden. Die primäre Untersuchung erfolgt im Closed-World, ergänzt um ein einzelnes Open-World-Experiment als Realitätsanker. Die formale Einordnung der Variablen und Messgrößen steht in Kapitel 4.

3.3 Wahl des Simulators

Die Umgebung muss vier Anforderungen erfüllen: realistische Verkehrsmuster, frei konfigurierbares Padding, exakte Reproduzierbarkeit und einen klar definierten Aufzeichnungspunkt. Emulationsbasierte Tor-Testumgebungen wie ExperimentTor und NetMirage verfolgen den Ansatz der Netzwerk-Emulation und nicht den für diese Arbeit benötigten Ansatz der reproduzierbaren, ereignisbasierten Simulation [1]. Live-Tor-Messungen sind nicht reproduzierbar [4]. Shadow mit Tornettools erfüllt alle Anforderungen: unmodifizierter Tor-Code und deterministischer Ablauf [19], und Jansen und Wails belegen mit 86 % Cross-Dataset-Accuracy des CUMUL-Klassifikators die Eignung für WF-Studien [4].

3.4 Wahl des Klassifikators

Aus den in Kapitel 2 verglichenen Klassifikatoren wird DF gewählt. DF arbeitet zeitunabhängig, weshalb simulationsbedingte Timing-Variationen die Ergebnisse nicht verfälschen. Circuit Padding wirkt zudem direkt auf die von DF genutzten

Richtungsfeatures, was den Padding-Effekt besonders sichtbar macht. Die Validierung von DF in Shadow durch Jansen und Wails (76 % bis 82 % Accuracy auf Live-Tor) sichert die Vergleichbarkeit mit bestehender Forschungsliteratur [4].

3.5 Wahl des Szenarios

Die primäre Untersuchung erfolgt im Closed-World, weil der Fokus auf dem isolierten Padding-Effekt liegt und nicht auf der Robustheit gegenüber unbekannten Seiten. Die Closed-World-Accuracy ist zudem deutlich stabiler als die Open-World-Kennzahlen [4], was beobachtete Effekte klarer dem Padding zuordnen lässt. Ergänzend wird ein Open-World-Experiment als Realitätsanker durchgeführt, ohne den Padding-Modus dort systematisch zu variieren.

3.6 Wahl der Padding-Konfigurationen

Der Padding-Modus nimmt drei Ausprägungen an: OFF, ON und Reduced. Diese Wahl orientiert sich an den von Tor selbst angebotenen Optionen und erlaubt einen direkten Vergleich mit dem von Kadianakis et al. vorgeschlagenen Schema [25]. Eine eigene Padding-Maschine wird nicht entwickelt, da das Ziel die Evaluation des produktiv eingesetzten Schemas ist und nicht der Entwurf einer neuen Verteidigung.

3.7 Wahl der Tamaraw-Integration

Tamaraw ist nicht Bestandteil des regulären Tor-Daemons. Im Gegensatz zum produktiv ausgerollten Circuit Padding plant das Tor-Projekt auch keine native Implementierung. Tamaraw verzögert reale Zellen systematisch und erreicht in der ursprünglichen Auswertung rund 199 % Bandbreiten-Overhead bei zusätzlichem Latenz-Overhead [24]. Beides widerspricht dem expliziten low-latency-Designziel von Tor, das eine Verzögerung realer Anwendungs-Zellen ausschliesst [25].

Um Tamaraw dennoch in einem Tor-Kontext zu untersuchen, wird das Verfahren als externes Hilfsprogramm zwischen Monitor und Bridge realisiert, das den Zellfluss vor dem Versand abfängt, gemäss Tamaraw-Regelwerk formt und anschliessend wieder an Tor übergibt. Die Einbindung erfolgt über Tors Pluggable-Transport-Schnittstelle¹. Ein Pluggable Transport (PT) ist ein eigenständiges Hilfsprogramm, das der Tor-Daemon vor sich schaltet. Jede Tor-Verbindung wird durch das Programm geleitet, das die Bytes vor dem Versand transformieren und beim Empfang zurücktransformieren kann. Diese Schnittstelle ist in der Praxis vor allem zur Zensurumgehung (etwa obfs4) im Einsatz, ist aber generisch genug, um beliebige Strom-Transformationen einzuhängen, darunter auch eine WF-Verteidigung.

¹<https://spec.torproject.org/pt-spec/>

3.8 Wahl des Hintergrundverkehrs

Der Hintergrundverkehr stammt aus den von Tormetools bereitgestellten TGen-Clients, die das Anwendungsverhalten realer Tor-Nutzung über Markov-Modelle nachbilden [1]. Eine eigene Verkehrsmodellierung wird nicht entwickelt, da die bestehenden Modelle bereits eine validierte Grundlage für realistischen Mischverkehr bilden.

3.9 Wahl der Inhaltsquelle

Als Inhalt für die zu klassifizierenden Webseiten wird die Simple-English-Wikipedia verwendet. Wikipedia liefert pro Artikel eine eigenständige HTML-Seite, was eine saubere Eins-zu-eins-Zuordnung zwischen Klasse und Inhalt ermöglicht. Da in Shadow kein Internetzugriff möglich ist, wird ein lokaler Spiegel eingerichtet.

3.10 Methodische Vereinfachung: PCAP statt Cell-Trace

Jansen und Wails verwenden eine gepatchte Tor-Version, die pro Zelle einen beschrifteten Eintrag mit Circuit-Zuordnung in einen Cell-Trace schreibt [4]. Diese Variante ist nicht öffentlich verfügbar.

Stattdessen wird in dieser Arbeit am Monitor eine fortlaufende PCAP-Datei mit allen TCP-Paketen aufgezeichnet und im Nachgang in beschriftete Richtungssequenzen pro Seitenbesuch zerlegt.

Während ein Cell-Trace die Pakete bereits zellgenau und mit Circuit-Information versehen liefert, ist das PCAP ein einziger unbeschrifteter Paketstrom, der erst nachträglich den einzelnen Besuchen zugeordnet werden muss. Diese Zerlegung übernimmt ein eigens für die vorliegende Arbeit entwickeltes Python-Skript (vgl. Abschnitt 5.5.1), das die Pakete anhand der in der `schedule.json` protokollierten Besuchsfenster und Labels (Abschnitt 3.12) den jeweiligen Seiten zuweist.

Damit arbeitet die vorliegende Pipeline auf TCP-Ebene und nimmt einen Detailverlust in Kauf. Die zellgenaue Auflösung wird nicht erreicht, was gegenüber Cell-Traces einen Informationsverlust bedeutet.

Zugleich bildet dieses PCAP-Verfahren ein realistisches Angriffsszenario eines passiven Beobachters ab, dem ebenfalls keine internen Tor-Cell-Traces zur Verfügung stehen [16].

Die Sequenzlänge wird nach Sirinam et al. auf 5000 Werte fixiert [2]. Jeder Wert entspricht in der vorliegenden Pipeline einem TCP-Paket innerhalb des Besuchsfensters, bei Sirinam et al. einer Tor-Zelle. Aufzeichnungen mit mehr als 5000 Paketen werden am Ende abgeschnitten, kürzere Aufzeichnungen mit Nullen aufgefüllt.

3.11 Merkmalswahl: nur Richtungen

Als Eingangsmerkmal werden ausschliesslich die Paketrichtungen verwendet (ausgehend $+1$, eingehend -1), keine Zeitstempel. Circuit Padding wirkt direkt auf die Richtungsabfolge, weshalb der Effekt im Richtungssignal am deutlichsten sichtbar wird.

3.12 Zuordnung von Verkehr zu Webseiten

Aus dem aufgezeichneten Paketstrom sollen beschriftete Trainingsbeispiele werden, jeweils bestehend aus einer Richtungssequenz und dem Label der abgerufenen Webseite. Dazu muss jeder Seitenbesuch zweifach identifiziert werden: zeitlich, um die Besuche im fortlaufenden Paketstrom voneinander zu trennen, und inhaltlich, um die jeweils abgerufene Webseite zu benennen.

Die zeitliche Trennung folgt aus dem festen Visit-Intervall (vgl. Abschnitt 3.14). Jeder Monitor startet pro Intervall genau einen Seitenabruf, sodass jedem Zeitfenster genau ein Besuch entspricht.

Die inhaltliche Zuordnung ist aus dem Monitor-PCAP allein nicht rekonstruierbar. Das PCAP zeichnet ausschliesslich den verschlüsselten Verkehr zum Tor-Guard auf, weder Ziel-IP noch Zielpport des angesprochenen Webservers sind sichtbar. Genau dieser Umstand entspricht dem realistischen Angriffsmodell: der passiven beobachtenden Instanz zwischen Monitor und Guard fehlt eben jene Information. Sie wird deshalb beim Aufbau der Simulation in einer Plandatei `schedule.json` (vgl. Abschnitt 5.4.2) festgehalten, die pro Monitor und Besuch die Startzeit und die abgerufene Seite enthält. Bei der Auswertung wird der PCAP-Strom an den Zeitgrenzen der Plandatei zerlegt und das hinterlegte Label übernommen. Eine Markierung im Tor-Datenstrom selbst oder eine gepatchte Tor-Version, wie sie Jansen und Wails einsetzen [4], wird damit umgangen.

3.13 Circuit-Isolation per NEWNYM

Vor jedem Seitenbesuch wird über das Tor-Control-Interface ein neuer Circuit erzwungen. Damit sind die einzelnen Besuche statistisch unabhängig und Verkehr aus früheren Besuchen kontaminiert die Zuordnung nicht. Diese strikte Isolation ist eine Vereinfachung gegenüber dem realen Tor-Browser-Verhalten (First-Party-Bundling) [4], in der WF-Forschungsliteratur jedoch übliche Praxis.

3.14 Round-Robin-Schedule

Die Besuche werden klassenübergreifend nach einem Round-Robin-Verfahren verteilt: zuerst jede Seite einmal, dann jede ein zweites Mal und so fort. Dadurch sind alle Klassen zeitlich gleichmässig über den Simulationszeitraum verteilt, was

systematische Korrelationen zwischen Klasse und Netzwerkzustand (Last, Cache-Effekte) verhindert. Zusätzlich bleibt die Klassen-Balance auch bei vorzeitigem Abbruch eines Laufs weitgehend erhalten.

3.15 Skalierung der Topologie

Die Tor-Topologie wird mit Tornettools auf 1 % des Live-Netzwerks skaliert [1], was in der verwendeten Januar-2025-Topologie 79 Relais ergibt. Eine Vollskalierung übersteigt den Arbeitsspeicher der bereitgestellten VM (128 GB) und macht die Simulationsdauer unpraktikabel für eine Bachelorarbeit. Die Reduktion bildet das Live-Netzwerk nur eingeschränkt ab, sodass sich das kleinere Netzwerk anders oder weniger realistisch verhalten kann und die absoluten Accuracy-Werte nicht unmittelbar auf das reale Netzwerk übertragbar sind. Die methodischen Konsequenzen dieser Reduktion werden in Kapitel 6 aufgegriffen.

4 Methoden

In diesem Kapitel wird beschrieben, mit welchen wissenschaftlichen Methoden die in Kapitel 2 aufgeworfenen Fragen bearbeitet werden. Zwei Methoden werden eingesetzt. Die Literaturrecherche dient dazu, den aktuellen Stand der Forschung zu Tor, Website-Fingerprinting und Circuit Padding aufzuarbeiten und daraus die Forschungslücke abzuleiten. Das anschließende Laborexperiment im Shadow-Simulator dient dazu, die aus dieser Forschungsliteratur motivierten Forschungsfragen quantitativ zu beantworten. Die konkrete Umsetzung dieses Versuchsaufbaus wird in Kapitel 5 beschrieben. Das vorliegende Kapitel beschränkt sich auf die methodische Begründung der Vorgehensweise.

4.1 Methodische Einordnung der Arbeit

Die vorliegende Bachelorarbeit ist eine Mischform aus einer konstruktiven und einer empirischen Arbeit [28]. Konstruktiv ist die Arbeit, weil eine eigene Werkzeugkette aus Shadow-Steuerung, Konfigurationsgenerierung, Webserver, Datenkonvertierung und Klassifikator-Anbindung neu aufgebaut wird, um den untersuchten Effekt überhaupt messen zu können. Empirisch ist die Arbeit, weil mit dieser Werkzeugkette ein kontrolliertes Laborexperiment durchgeführt wird, dessen Ergebnisse die in der Aufgabenstellung definierten Forschungsfragen beantworten.

Die Arbeit verwendet ein deduktives Vorgehen. Aus der Forschungsliteratur, insbesondere aus den Arbeiten von Jansen und Wails [4], Sirinam et al. [2] und Kadianakis et al. [25], wird der theoretische Rahmen abgeleitet, in dem die Forschungsfragen aus Kapitel 1 bearbeitet werden. Die Messung selbst erfolgt quantitativ über etablierte Klassifikationsmetriken (Accuracy, Precision, Recall, F1-Wert).

4.2 Literaturrecherche

Die Literaturrecherche folgt dem in Balzert beschriebenen fünfstufigen Vorgehen aus den Schritten was suche ich, wo suche ich, wie suche ich, was verwende ich, wie beschaffe ich [28]. Die einzelnen Stufen werden im Folgenden zusammengefasst.

4.2.1 Suchstrategie und Abgrenzung des Themenfelds

Ausgangspunkt der Recherche ist das in Kapitel 1 formulierte Forschungsthema rund um den Einfluss von Circuit Padding auf Website-Fingerprinting in Shadow. Aus dem Themenfeld werden vier inhaltliche Suchrichtungen abgeleitet: das Tor-Netzwerk und sein Bedrohungsmodell, Website-Fingerprinting-Klassifikatoren, Circuit Padding als Verteidigung sowie der Shadow-Simulator und die zugehörige Werkzeugkette `tornettools`.

4.2.2 Genutzte Quellen, Datenbanken und Suchmaschinen

Die Recherche stützt sich primär auf wissenschaftliche Datenbanken und nicht auf allgemeine Websuchmaschinen. Folgende Quellen werden eingesetzt:

- Google Scholar als übergreifende Suchmaschine, die zusätzlich die Vorwärts-Zitationen eines Artikels über die Funktion Cited by erschliesst.
- swisscovery als Bibliothekskatalog der HSLU für Lehrbücher, insbesondere für die methodologische Grundlage [28].
- Primärquellen wie das offizielle Tor-Spezifikationsdokument [8], der Tor-Metrics-Datendienst [20] und das OnionPerf-Projekt [29].

4.2.3 Bewertung der Quellen

Jede gefundene Quelle wird nach den beiden Balzert-Kriterien Zitierfähigkeit und Zitierwürdigkeit geprüft [28]. Zitierfähig sind Quellen, die veröffentlicht, nachvollziehbar und durch Dritte kontrollierbar sind. Zitierwürdig sind Quellen, die einen wissenschaftlichen Anspruch erfüllen, also peer-reviewed sind oder von einer anerkannten Institution stammen, im Gegensatz zu populärwissenschaftlichen oder werblichen Texten. Primärquellen werden gegenüber Sekundärquellen bevorzugt, weshalb beispielsweise die Tor-Spezifikationen [8] direkt herangezogen werden und nicht die zusammenfassende Tertiärliteratur.

4.2.4 Literaturverwaltung

Sämtliche während der Recherche ermittelten Quellen werden in einem zentralen BibTeX-Verzeichnis (`src/thesis/references.bib`) gepflegt. Pro Quelle werden Autorenschaft, Titel, Erscheinungsjahr, Verlag oder Konferenz, Digital Object Identifier (DOI) oder URL und das Zugriffsdatum erfasst. Im Text wird nach IEEE-Zitierstil zitiert, wie in der HSLU Vorlage vorgegeben.

4.3 Experimentelles Vorgehen

Die zentrale empirische Methode dieser Arbeit ist das kontrollierte Laborexperiment. Dieser Abschnitt beschreibt das experimentelle Design auf methodischer Ebene. Die konkrete technische Umsetzung folgt in Kapitel 5.

4.3.1 Wahl des Experimenttyps

Aus den von Balzert beschriebenen Experimenttypen Laborexperiment und Feldexperiment [28] fällt die Wahl auf das Laborexperiment in einer deterministischen Netzwerksimulation. Ein Feldexperiment im Live-Tor-Netzwerk scheidet aus, weil dort weder die Padding-Konfiguration der Relays kontrolliert variiert werden kann noch die Erzeugung grosser Mengen Trainingsverkehrs ethisch vertretbar wäre. Der

Shadow-Simulator erlaubt im Gegensatz dazu, die unabhängige Variable Padding-Modus gezielt zu schalten, alle übrigen Parameter konstant zu halten und denselben Versuch über einen festen Seed reproduzierbar zu wiederholen. Damit erfüllt das Vorgehen die für ein Laborexperiment gegebenen Anforderungen an Kontrolle, Wiederholbarkeit und Variation einer Einflussgrösse [28].

4.3.2 Variablen und Messgrössen

Das Experiment unterscheidet zwischen unabhängigen und abhängigen Variablen, wie es für ein kontrolliertes Laborexperiment gefordert ist [28].

Unabhängige Variablen Drei Variablen werden gezielt variiert. Erstens das Verteidigungsverfahren mit den drei in Tor angebotenen Padding-Modi OFF, ON und Reduced (vgl. Tabelle 5.2, Kapitel 5) sowie Tamaraw als ergänzendes, alternatives Schema. Zweitens die Anzahl überwachter Webseiten mit den Ausprägungen 20, 80 und Open-World (80 überwachte Webseiten in einem Pool von 280 Quellseiten, entsprechend 81 effektiven Klassen). Drittens das Welt-Szenario mit den Ausprägungen Closed-World und Open-World.

Abhängige Variablen Gemessen werden Accuracy, Precision, Recall und F1-Wert auf einer zurückgehaltenen Testmenge. Diese vier Klassifikationsmetriken sind in der WF-Forschungsliteratur etabliert und ergänzen sich gegenseitig. Sie werden aus den vier Grundgrössen True Positive (TP, korrekt als überwacht klassifiziert), True Negative (TN, korrekt als unüberwacht klassifiziert), False Positive (FP, fälschlich als überwacht klassifiziert) und False Negative (FN, fälschlich als unüberwacht klassifiziert) berechnet:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4.2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4.3)$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.4)$$

Accuracy gibt den Anteil korrekt klassifizierter Beispiele an und ist bei balancierten Klassen aussagekräftig, was dem Closed-World-Szenario dieser Arbeit entspricht. Precision misst den Anteil tatsächlich überwachter Seiten unter den als überwacht klassifizierten und ist im Open-World-Szenario relevant, weil dort eine niedrige False-Positive-Rate gefordert ist. Recall misst den Anteil der erkannten überwachten Seiten an allen tatsächlich überwachten und beschreibt die Erfassungsrate der angreifenden Partei. Der F1-Wert ist das harmonische Mittel aus Precision und Recall und bleibt auch bei stark unbalancierten Klassen wie im Open-World-Szenario ein robustes Mass.

Konstant gehaltene Rahmenbedingungen Alle übrigen Parameter bleiben über die Experimente einer Vergleichsgruppe konstant. Dazu zählen die Skalierung des simulierten Netzwerks (1 %), der Simulator-Seed, der Seed der Trainings- und Testaufteilung, die Hyperparameter des Klassifikators sowie die Auswahl der Wikipedia-URLs.

4.3.3 Versuchsplan und Ausführungsreihenfolge

Die Durchführung erfolgt in vier aufeinander aufbauenden Stufen. In einer ersten Stufe wird die Werkzeugkette mit den kleinen Closed-World-Experimenten validiert, weil deren kurze Laufzeit ein schnelles Feedback zu Fehlern in Simulation, Datenkonvertierung und Klassifikator-Anbindung erlaubt. In einer zweiten Stufe werden dieselben Padding-Modi auf die grössere Closed-World übertragen, um den Effekt unter realistischeren Bedingungen zu prüfen. In einer dritten Stufe wird das Open-World-Szenario gemessen, ebenfalls über alle drei Padding-Modi, um den Padding-Effekt auch mit nicht überwachten Webseiten zu prüfen. In einer vierten Stufe wird ergänzend Tamaraw als alternatives, nicht implementiertes Verteidigungsverfahren über die drei Konfigurationen 20, 80 und Open-World (80 überwachte Webseiten aus 280 Quellseiten) gemessen. Jeder Durchlauf besteht aus den Schritten Simulation in Shadow, Konvertierung der PCAP-Aufzeichnungen in das Wfrib-Format und Training sowie Test des DF-Klassifikators auf einer festen Datenaufteilung.

4.3.4 Grenzen der gewählten Methode

Die Methode hat zwei strukturelle Grenzen: Erstens ist das simulierte Netzwerk mit 1 % der Live-Grösse deutlich kleiner als das reale Tor-Netzwerk und kann sich dadurch anders oder weniger realistisch verhalten, weshalb die absoluten Accuracy-Werte nicht unmittelbar auf das Live-Netzwerk übertragbar sind. Zweitens wird der Tor Browser durch ein gepatchtes `wget2` ersetzt, das zwar denselben Satz an Ressourcen abrufen, jedoch JavaScript und Browser-spezifische Verkehrsmuster nicht reproduziert [4]. Beide Einschränkungen werden bei der Interpretation der Resultate in Kapitel 6 berücksichtigt.

4.4 Einsatz von KI-Werkzeugen

Dieser Abschnitt dokumentiert, welche KI-gestützten Werkzeuge im Rahmen dieser Arbeit eingesetzt wurden, wofür sie verwendet wurden und wie die fachliche Verantwortung für die vorliegende Arbeit gewahrt bleibt. Die Darstellung ist als Ergänzung zur Selbstständigkeitserklärung dieser Arbeit zu verstehen.

4.4.1 Eingesetzte Werkzeuge

Tabelle 4.1 listet die verwendeten KI-Werkzeuge, die jeweils genutzten Modellversionen und die konkreten Anwendungsbereiche auf.

Tabelle 4.1: Im Rahmen der Arbeit eingesetzte KI-Werkzeuge.

Werkzeug	Modell(e)		Verwendung
Claude Code	Opus 4.5, 4.6, Opus 4.8	Opus 4.7,	Hauptsächlich genutztes Werkzeug. Iterative Entwicklung sämtlicher in dieser Arbeit selbst geschriebenen Python- und Shell-Skripte.
Claude Desktop	Opus 4.5, 4.6, Opus 4.8	Opus 4.7,	Sprachliche Politur, Vereinheitlichung des Stils, Korrektur von Formulierungen und Tippfehlern an Teilen des Prosatexts der Arbeit.

4.4.2 Art der Nutzung

Code Die Skripte wurden nicht in einem einzigen Schritt von einer KI generiert, sondern in einem iterativen Dialog entwickelt. Zunächst wurde die fachliche Anforderung formuliert (Eingabe, Ausgabe, Bibliotheken, Constraints). Claude Code generierte daraufhin einen Implementierungsentwurf, der anschliessend gelesen, geprüft und korrigiert wurde. Danach wurde das Skript ausgeführt und gegen die erwarteten Resultate validiert. Als hilfreich hat sich die Planungsfunktion erwiesen, bei welcher die KI einen Plan erstellt, bevor sie Code schreibt. So können Fehler und Abweichungen früh korrigiert werden.

Text Der Prosatext wurde teilweise mit KI-Werkzeugen sprachlich überarbeitet.

4.5 Projektplan

Abbildung 4.1 zeigt den zeitlichen Verlauf der Arbeit von Semesterbeginn am 16. Februar 2026 bis zur Abgabe am 15. Juni 2026. Die Meilensteine (rot) markieren fixe Termine, die Balken die fachlichen Arbeitsphasen.

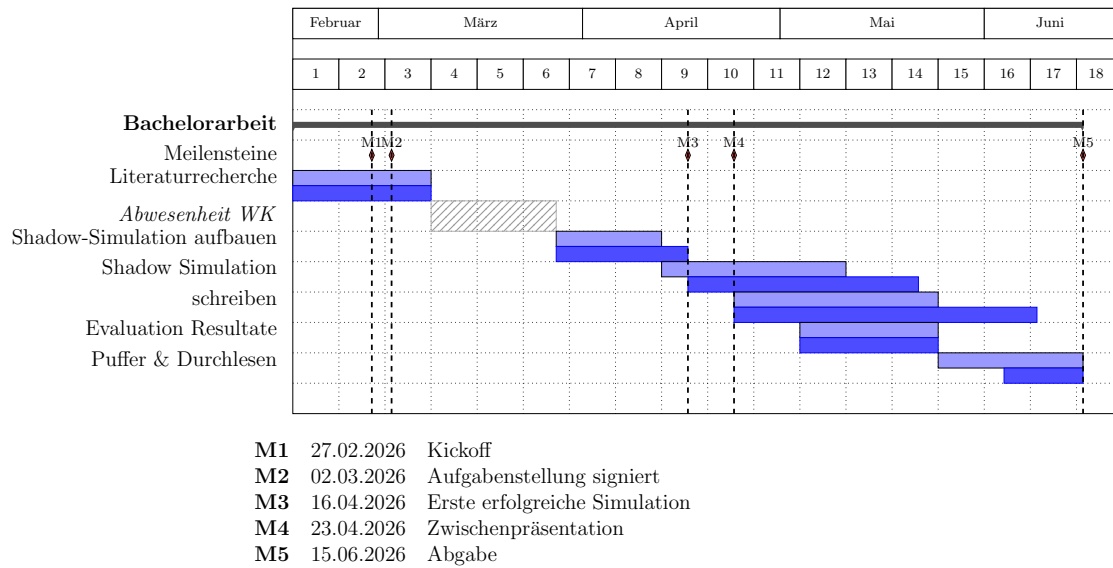


Abbildung 4.1: Projektplan der Bachelorarbeit mit Arbeitsphasen (blau) und fixen Meilensteinen.

5 Realisierung

5.1 Übersicht des Versuchsaufbaus

Der Versuchsaufbau besteht aus drei aufeinander aufbauenden Teilsystemen. Zuerst wird mit dem Shadow-Simulator [19] ein realitätsnahes Tor-Netzwerk auf einem einzelnen Server simuliert. Innerhalb dieses simulierten Netzwerks führt eine Anzahl von Monitor-Knoten automatisierte Webseitenbesuche über Tor durch. Der dabei anfallende Netzwerkverkehr wird auf den Monitor-Knoten als PCAP-Datei mitgeschnitten. In einer zweiten Stufe werden diese Aufzeichnungen exportiert und in ein passendes Dateiformat konvertiert. In der dritten Stufe wird auf den Daten ein DF-Klassifikator trainiert und ausgewertet.

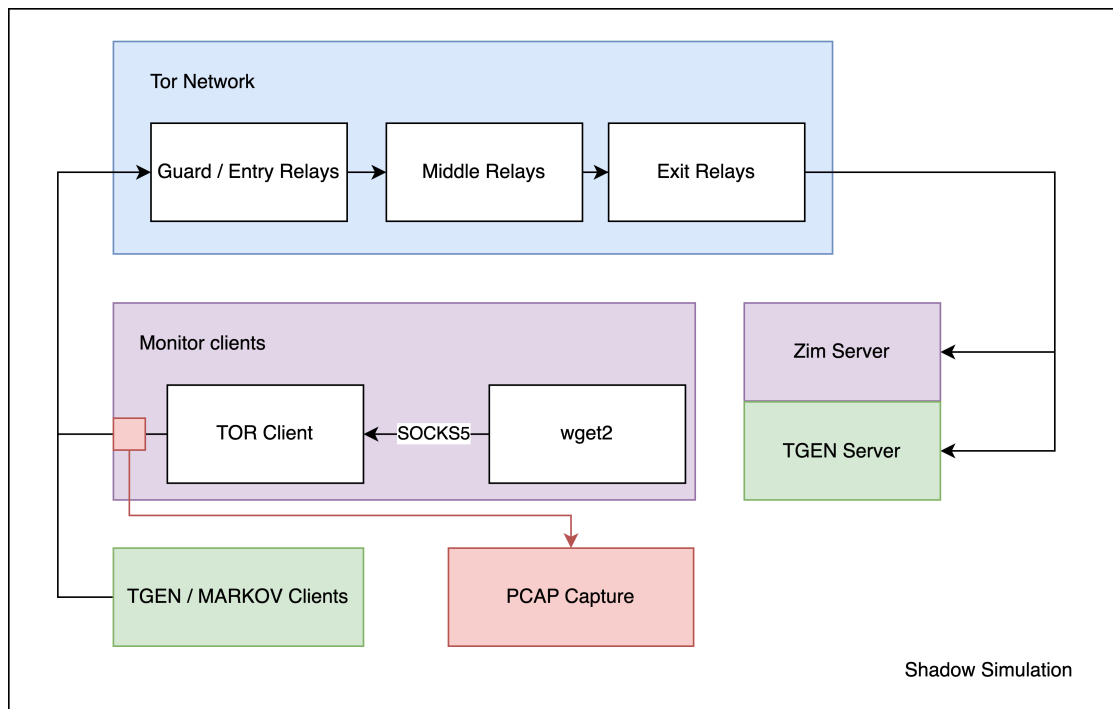


Abbildung 5.1: Innerhalb der Simulation existiert ein vollständiges Tor-Netzwerk mit Eingangs-, Mittel- sowie Ausgangsknoten. Daneben laufen die für diese Arbeit eigens hinzugefügten Monitor-Clients (jeweils mit lokalem Tor-Daemon, gepatchtem `wget2` und PCAP-Aufzeichnung), ein Zim-Server für die Wikipedia-Inhalte sowie die von Tornettools generierten TGEN- und Markov-Hintergrund-Clients, deren Verkehr das Verhalten realer Tor-Nutzender nachbildet [1].

Der gesamte Ablauf wird von dem zentralen Steuerungsskript `shadowctl.py` orchestriert, das per Secure Shell (SSH) auf den Simulationsserver zugreift, dort die Simulation startet, ihren Fortschritt überwacht und nach dem Lauf die Ergebnisdateien zurück auf die lokale Arbeitsstation überträgt. Diese Trennung zwi-

schen Steuerung (lokal) und Ausführung (remote) erlaubt eine benutzerfreundliche Steuerung der Simulationen.

5.2 Versuchsumgebung

5.2.1 Server

Sämtliche Simulationen laufen auf einer dedizierten virtuellen Maschine mit dem Hostnamen `shadowsrv-001.prod.projects.ls.eee.intern`, die von HSLU Lab Services für die Dauer dieser Arbeit bereitgestellt wurde. Die VM verfügt über 32 vCPU-Kerne, 128 GB Arbeitsspeicher und einen 156 GB grossen Speicher. Eine ausführliche Auflistung der bereitgestellten Ressourcen sowie der eingerichteten Software-Umgebung findet sich in Anhang B.

Der Zugriff auf die VM ist nur aus dem HSLU-Netzwerk (Campus-WLAN oder VPN) über SSH möglich, ein direkter Zugriff aus dem Internet ist administrativ unterbunden. Diese Einschränkung ist für den eigentlichen Versuch unerheblich, da Shadow seine Simulationen ausschliesslich lokal auf der VM ausführt und keine Aussenkommunikation benötigt. Sie ist jedoch für die Werkzeugkette relevant, weil das Steuerungsskript `shadowctl.py` per SSH mit der VM kommuniziert und damit nur aus dem HSLU-Netzwerk heraus funktioniert.

5.2.2 Werkzeugkette

Die Werkzeugkette gliedert sich in extern bezogene Komponenten und eigens für diese Arbeit entwickelte Skripte. Als externe Komponenten kommen zum Einsatz:

- Shadow [3, 19] simuliert das Tor-Netzwerk.
- Tornettools [1] erzeugt aus den öffentlichen Tor-Metrik-Daten eine repräsentative Shadow-Konfiguration.
- Der Wikipedia-ZIM-Dump dient als Inhalt für die zu klassifizierenden Webseiten.
- `wget2` mit dem SOCKS-Patch von Jansen und Wails [4] lädt im Inneren der Simulation die Webseiten über Tor herunter.
- `WFlib`¹ stellt eine Implementierung des DF-Klassifikators in PyTorch bereit und steuert Training, Validierung und Test.

Die eigens entwickelten Skripte sind in Tabelle 5.1 mit Aufgabe sowie ihren Ein- und Ausgaben zusammengefasst.

¹<https://github.com/Xinhao-Deng/Website-Fingerprinting-Library>

Tabelle 5.1: Eigens entwickelte Pipeline-Skripte mit ihrer jeweiligen Aufgabe.

Skript	Aufgabe
<code>shadowctl.py</code>	Steuerung der Experimente
<code>generate-wf-config.py</code>	Monitore und Webserver in shadow config eintragen
<code>generate-urls.py</code>	Zufallsstichprobe aus URL Liste generieren
<code>zimsrv.py</code>	Wikipedia Server in der Simulation
<code>newnym.py</code>	Frischen Tor-Circuit erzwingen
<code>pcap_to_npz.py</code>	Pcaps in WFlib-Format konvertieren
<code>run_df.sh</code>	Training und Auswertung steuern
<code>report.py</code>	Ergebnis-JSON aggregieren

Die Aufteilung in mehrere kleine, voneinander unabhängige Skripte erlaubt es, einzelne Phasen der Pipeline gezielt erneut auszuführen, ohne den zeitintensiven Simulationslauf zu wiederholen.

5.2.3 Generierung der Topologie mit Tornettools

Die Shadow-Konfiguration wird mit Tornettools aus den Tor-Metriken (Konsensdokumente, Server-Deskriptoren, OnionPerf-Messungen, Bandbreitenwerte) eines frei wählbaren Monats erzeugt [1]. In der vorliegenden Arbeit wird durchgehend der Monat Januar 2025 verwendet, die Standardeinstellung der Steuerungsskripte.

Die Tor-Netzwerkgrösse wird mit Tornettools auf 1 % (`--network_scale 0.01`) skaliert [1], was in der verwendeten Januar-2025-Topologie 79 Relais, drei Verzeichnisserver und rund 100 Hintergrund-Clients ergibt. Die Begründung der Skalierung und ihre methodischen Konsequenzen sind in Abschnitt 3.15 dargestellt.

5.2.4 Erweiterung der Konfiguration für das Fingerprinting

Die von Tornettools erzeugte Konfiguration enthält ausschliesslich Tor-spezifische Komponenten. Für ein WF-Experiment wird zusätzlich ein Webserver, der die Zielseiten ausliefert, und eine Anzahl von Monitor-Knoten benötigt, welche diese Seiten über Tor besuchen. Beides wird in dieser Arbeit von dem Skript `generate-wf-config.py` ergänzt.

Hinzufügen der Hosts `generate-wf-config.py` fügt der `shadow.config.yaml` einen `zimserverN`-Host und eine konfigurierbare Anzahl `monitorN`-Hosts hinzu. Pro besuchter Seite wird ein eigener Port auf dem Webserver verwendet.

Listing 5.1 zeigt einen verkürzten Ausschnitt eines Monitor-Hosts aus der generierten `shadow.config.yaml`.

```

monitor0:
  bandwidth_down: 100 megabit
  bandwidth_up: 100 megabit
  host_options:
    pcap_enabled: true
    pcap_capture_size: 65535 B
  processes:
    - path: ~/.local/bin/tor
      args: --defaults-torrc torrc-defaults -f torrc
      start_time: 240
      expected_final_state: running
    - path: /home/projectadmin/wget2_noinstall
      args: --page-requisites --max-threads=30 ...
          http://129.114.108.192:8000/Eurovision_Song_Contest_2018
      start_time: 1200
    - path: /usr/bin/python3
      args: /home/projectadmin/newnym.py
      start_time: 1229
    - path: /home/projectadmin/wget2_noinstall
      args: --page-requisites --max-threads=30 ...
          http://129.114.108.192:8010/Ionizing_radiation
      start_time: 1230

```

Listing 5.1: Auszug aus `shadow.config.yaml` für `monitor0`: aktivierte PCAP-Aufzeichnung sowie die Prozessabfolge Tor-Daemon, `wget2`-Aufruf und `newnym.py` (gekürzt auf zwei aufeinanderfolgende Besuche).

Schedule Zusätzlich erzeugt `generate-wf-config.py` eine Datei `schedule.json`, in der für jeden Monitor festgehalten ist, wann er welche Seite besucht. Diese Datei wird später für den Konvertierungsschritt genutzt: ohne sie liesse sich nicht eindeutig rekonstruieren, welcher Teil eines Pcaps zu welchem Seitenaufruf gehört. Ein Auszug des Dateiformats findet sich in Anhang D.1.

Padding-Konfiguration Die Padding-Parameter werden ebenfalls von `generate-wf-config.py` in die Shadow Konfiguration der Monitor-Hosts geschrieben. Tabelle 5.2 listet die drei in dieser Arbeit verwendeten Konfigurationen auf.

Tabelle 5.2: Drei untersuchte Padding-Konfigurationen.

Modus	torrc-Eintrag
OFF	CircuitPadding 0
ON	CircuitPadding 1
Reduced	CircuitPadding 1
	ReducedCircuitPadding 1

Die Begründung der Wahl genau dieser drei Modi ist in Abschnitt 3.6 dargestellt.

5.3 Bereitstellung der Webinhalte

5.3.1 Wikipedia als Datenquelle

Für die Webseiten wird die einfache englische Variante von Wikipedia in einer ZIM-Datei verwendet. ZIM ist ein offenes, vom Kiwix-Projekt definiertes Containerformat, das eine vollständige Webseiten-Sammlung in einer einzigen Datei zusammenfasst und Offline-Zugriff erlaubt².

Verwendet wird in dieser Arbeit ausschliesslich `wikipedia_en_simple_all_maxi`³. Diese Variante enthält Bilder und damit deutlich vielfältigere Webseiten als die bildlose Version, ist aber gleichzeitig klein genug, um vollständig in den Speicher des Servers zu passen.

Tabelle 5.3: Varianten der Wikipedia-ZIM-Datei (Stand 2026-02).

Datei	Grösse	Bemerkung
<code>wikipedia_en_simple_all_nopic</code>	921 MB	Einfaches Englisch ohne Bilder
<code>wikipedia_en_simple_all_maxi</code>	3.2 GB	Einfaches Englisch mit Bildern
<code>wikipedia_en_all_maxi</code>	115 GB	Vollständiges englisches Wikipedia

5.3.2 Webserver: `zimsrv.py`

Für den Webserver wurden mehrere Varianten geprüft.

ZIMply In der Arbeit von Jansen und Wails wird ZIMply als Webserver für ZIM-Dateien verwendet [4]. Die Bibliothek wurde schon länger nicht mehr aktualisiert und funktionierte in der Simulation nicht zuverlässig.

kiwix-serve Kiwix-Serve ist die offizielle Referenzimplementierung des Kiwix-Projekts⁴ und wäre funktional die ideale Wahl. Es wird jedoch als statisch gelinkte Binärdatei ausgeliefert. Eine statisch gelinkte Binärdatei kann von Shadow nicht ausgeführt werden, was den Einsatz erschwert.

zimsrv.py Aus diesen Gründen wurde im Rahmen dieser Arbeit ein eigener, schlanker Webserver `zimsrv.py` mit etwa 125 Zeilen Python entwickelt. Die Implementierung erfolgte iterativ mit Claude Code (vgl. Abschnitt 4.4). Da der Server in Python läuft, ist er per Definition dynamisch gelinkt und damit für Shadow geeignet. Pro Seite wird ein separater Prozess gestartet.

²<https://kiwix.org> (Zugriff: 2026-06-05).

³ZIM-Dateien: <https://download.kiwix.org/zim/wikipedia/> (Zugriff: 2026-04-05).

⁴<https://github.com/kiwix/kiwix-tools> (Zugriff: 2026-06-05).

5.4 Monitor-Knoten und Verkehrserfassung

5.4.1 Webclient: gepatchtes wget2

Der Tor Browser kann in Shadow nicht ausgeführt werden, da er auf eine grafische Oberfläche und JavaScript-Engines angewiesen ist. In der vorliegenden Arbeit wird der von Jansen und Wails entwickelte SOCKS-Patch für `wget2` verwendet (vgl. Abschnitt 2.4.6). Er wird aus dem öffentlichen Artefakt-Repository⁵ bezogen, auf einen festgehaltenen `wget2`-Commit angewendet und einmalig auf dem Server gebaut⁶.

5.4.2 Besuchsplanung und Round-Robin-Verteilung

Die Seiten werden nach dem in Abschnitt 3.14 beschriebenen Round-Robin-Verfahren auf die Monitore verteilt.

Zwischen zwei Besuchen liegen 30 Sekunden, ausreichend auch für grössere Wikipedia-Seiten samt eingebetteter Bilder. Der erste Besuch beginnt nach einer Aufwärmphase von 1200 Sekunden (20 Minuten), in der das Tor-Netzwerk seinen Konsens aushandelt und die Markov-Hintergrundclients ihre Verkehrsmuster aufbauen.

5.4.3 Circuit-Isolation per NEWNYM

Damit jeder Seitenbesuch über einen frischen Circuit läuft, plant `generate-wf-config.py` jeweils eine Sekunde vor einem `wget2`-Aufruf einen `newnym.py`-Prozess in der `shadow.config.yaml` ein. Der vollständige Quelltext des Skripts ist in Anhang D.3 abgedruckt.

Die Begründung dieser strikten Circuit-Isolation ist in Abschnitt 3.13 dargestellt.

5.4.4 Aufzeichnung des Datenverkehrs

Auf jedem Monitor-Host wird über das Shadow Konfigurationsfeld `host_options.pcap_enabled` der Netzwerkverkehr aufgezeichnet.

Die so aufgezeichneten Pcaps werden im Anschluss an die Simulation per `shadowctl.py pull-results` auf die Arbeitsstation des Autors übertragen.

⁵<https://github.com/explainwf-popets2023/explainwf-popets2023.github.io> (Zugriff: 2026-06-05).

⁶`setup-wf-server.sh`

5.5 Aufbereitung der Daten für die Klassifikation

5.5.1 Konvertierung der Pcaps in das WFlib-Format

Das Skript `pcap_to_npz.py` überführt die Monitor-Pcaps zusammen mit der `schedule.json` in das von WFlib erwartete `.npz`-Format. Pro Experiment entstehen eine `.npz`-Datei mit den Richtungssequenzen sowie eine `labels.json`-Datei mit der Zuordnung von Seite zu Klassen-Label (Abbildung 5.2).

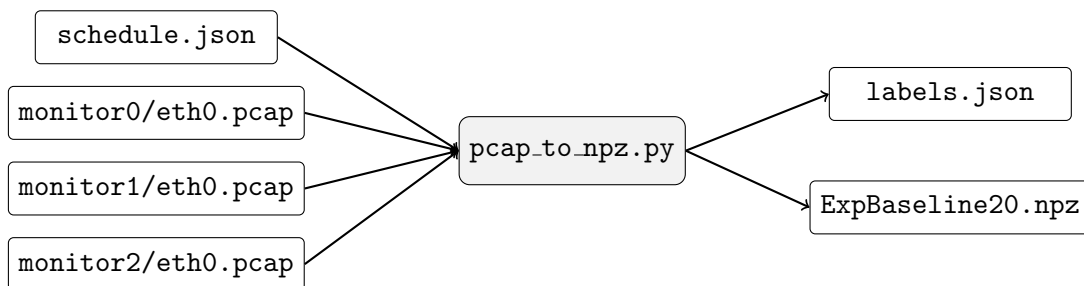


Abbildung 5.2: Konvertierungs-Pipeline: `pcap_to_npz.py` liest die Monitor-Pcaps und die `schedule.json`, schneidet pro Besuch die Paketrichtungen aus und schreibt sie zusammen mit den Klassen-Labels in eine kompakte `.npz`-Datei.

Für jeden Eintrag der `schedule.json` wird das Zeitintervall des Besuchs aus dem PCAP geschnitten, Pakete mit der Client-IP als Quelle gelten als ausgehend (+1), Pakete mit der Client-IP als Ziel als eingehend (−1). Die Klassen-Labels werden aus der `schedule.json` übernommen und zusammen mit der Umkehrabbildung und der Anzahl der Klassen als `labels.json` neben der `.npz`-Datei abgelegt, damit die Ergebnisse später eindeutig den ursprünglichen Wikipedia-Seiten zugeordnet werden können.

5.5.2 Aufteilung in Trainings-, Validierungs- und Testdatensatz

Die WFlib-Komponente `dataset_split.py` teilt den Datensatz in 81 % Training, 9 % Validierung und 10 % Test auf. Die Aufteilung erfolgt zufällig, durch einen festen Zufalls-Seed wird sie aber reproduzierbar: derselbe Datensatz wird bei jedem Aufruf identisch zerlegt.

5.6 Training und Auswertung des Klassifikators

5.6.1 Bibliothek

Wie in Abschnitt 3.4 begründet, wird DF nach Sirinam et al. [2] als Klassifikator verwendet. Die Implementierung stammt aus WFlib, einer in PyTorch realisierten

Sammlung von WF-Klassifikatoren von Deng et al.⁷. WFLib hat sich in mehreren neueren Veröffentlichungen als Vergleichsplattform etabliert, was auch die Vergleichbarkeit der Ergebnisse erhöht.

5.6.2 Hyperparameter

Diese Trainingsbedingungen bleiben über alle Experimente hinweg identisch. Beobachtete Genauigkeitsunterschiede sind ausschliesslich auf die unabhängigen Variablen (Padding-Modus und Klassenanzahl) zurückzuführen, nicht auf abweichende Modelleinstellungen. Die Werte sind in Tabelle 5.4 zusammengefasst.

Tabelle 5.4: Hyperparameter des DF-Trainings.

Parameter	Wert
Eingangsmerkmal	DIR (Richtung, ± 1)
Sequenzlänge	5000
Optimierer	Adamax
Lernrate	$2 \cdot 10^{-3}$
Verlustfunktion	Cross-Entropy
Batchgrösse	128
Epochen	30

Während des Trainings passt der Adamax-Optimierer die Gewichte des neuronalen Netzes Schritt für Schritt an, um eine möglichst geringe Fehlerrate zu erreichen. Dabei speichert der Optimierer nach jeder Epoche das Modell, sofern es auf der Validierungsmenge einen besseren F1-Wert erreicht als alle bisherigen Checkpoints. Damit ist das endgültig getestete Modell nicht zwangsläufig das letzte, sondern das auf Basis der Validierung beste.

5.6.3 Trainingsablauf

Der gesamte Ablauf von der Konvertierung bis zur Auswertung läuft in einer einzigen Shell-Pipeline ab⁸.

1. Konvertierung der Pcaps in .npz Datei
2. Aufteilung in Trainings-, Validierungs- und Testdatensatz.
3. Training des DF-Modells über 30 Epochen.
4. Test auf der zurückgehaltenen Testmenge.
5. Speichern der Metriken in einer Ergebnis-JSON-Datei.

Pro Trainingslauf werden Accuracy, Precision, Recall und F1-Wert auf der Testmenge ermittelt und in einer Ergebnis-JSON abgelegt. Diese vier Kennzahlen werden mit den Metadaten des Experiments (Anzahl Klassen, Padding-Modus, Anzahl

⁷<https://github.com/Xinhao-Deng/Website-Fingerprinting-Library>

⁸`run_df.sh`

Besuche, Anzahl Monitore, Random-Baseline) angereichert und in die zentrale Ergebnistabelle `ExperimentLogs.csv` geschrieben⁹. Diese Tabelle dient als Single Source of Truth für alle in Kapitel 6 präsentierten Diagramme.

5.7 Tamaraw-Variante des Versuchsaufbaus

Für die ergänzende Messreihe gegen das Tamaraw-Verfahren (Abschnitt 2.5.4) muss der bisher beschriebene Versuchsaufbau um einen zusätzlichen Bridge-Knoten erweitert werden. Die übrigen Bestandteile, also Tor-Netzwerk aus Tornettools, Monitor-Clients, Zim-Server und PCAP-Aufzeichnung, bleiben unverändert. Abbildung 5.3 zeigt die so erweiterte Topologie.

⁹`report.py`

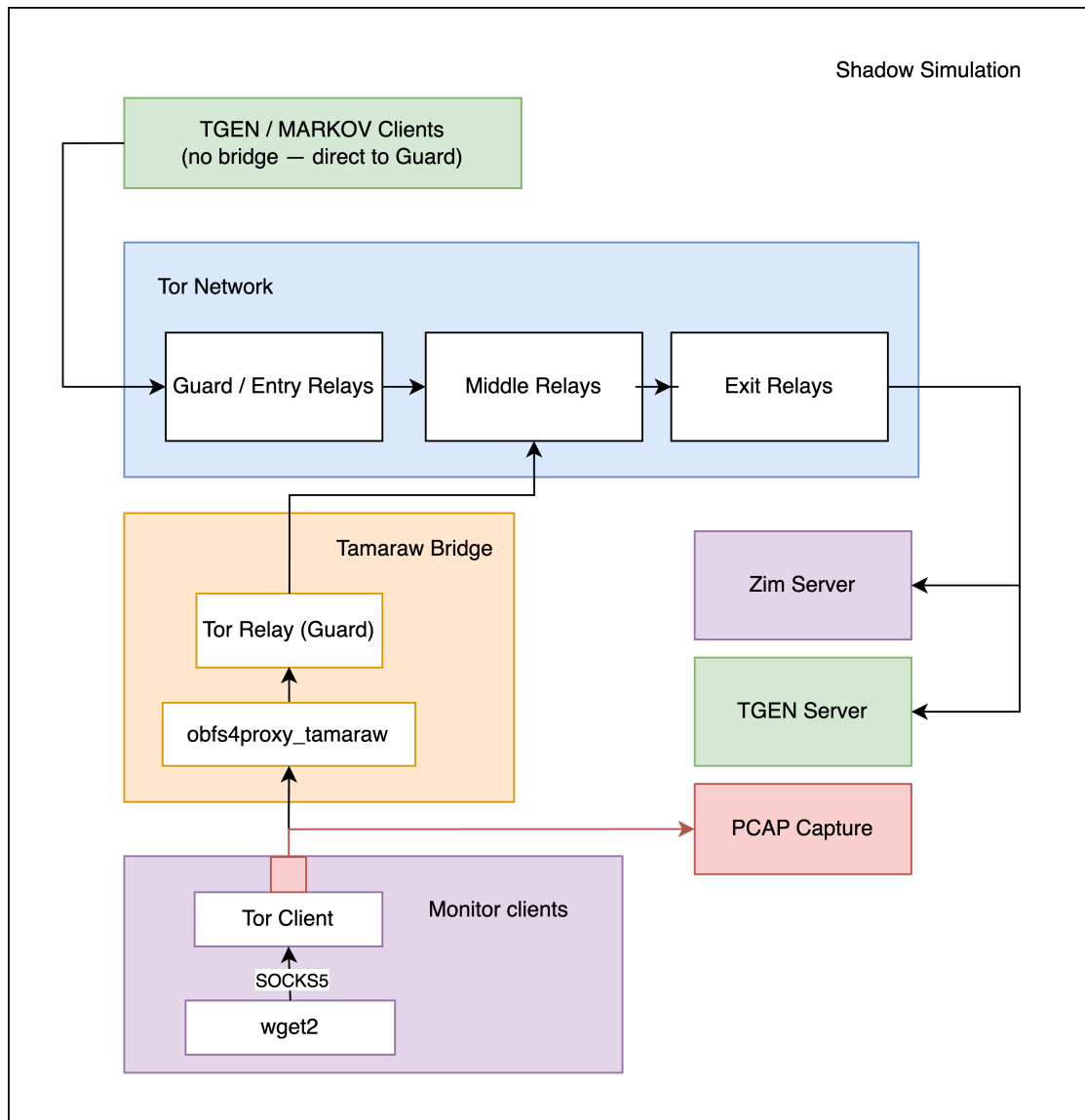


Abbildung 5.3: Tamaraw-Variante des Versuchsaufbaus. Gegenüber der Baseline-Topologie aus Abbildung 5.1 kommt der orange dargestellte Bridge-Knoten `wf-bridge0` hinzu. Der monitor-seitige Tor-Daemon verbindet sich nicht direkt zu einem Guard, sondern über den Bridge-Knoten, dessen Relay-Prozess die geformten Zellen anschliessend regulär in das simulierte Tor-Netzwerk einspeist.

5.7.1 Bridge-Knoten

Wie in Abschnitt 3.7 begründet, wird Tamaraw als PT realisiert. Dazu wird der Topologie ein zusätzlicher Bridge-Host hinzugefügt, auf dem zwei Prozesse parallel laufen. Der PT formt den Zellfluss in Richtung Monitor nach dem Tamaraw-Regelwerk. Der lokal mitlaufende Tor-Daemon übernimmt die Guard-Rolle für den Monitor-Circuit und leitet die geformten Zellen anschliessend regulär in das simulierte Tor-Netzwerk weiter.

5.7.2 Implementierung: WFDefProxy

Als konkrete Tamaraw-Implementierung wird WFDefProxy verwendet, eine quelloffene Sammlung von Website-Fingerprinting-Verteidigungen aus dem öffentlichen Artefakt-Repository¹⁰ [30]. WFDefProxy baut auf dem etablierten PT `obfs4proxy` auf und stellt unter anderem Tamaraw als auswählbare Verteidigung bereit. Das Werkzeug wird einmalig auf dem Server gebaut und übernimmt im Versuch die in Abschnitt 5.7.1 beschriebene Rolle des PT.

Die Tamaraw-Parameter werden in den Voreinstellungen von WFDefProxy belassen.

5.7.3 Einbindung in die Shadow-Konfiguration

Die Einbindung in die Shadow-Topologie übernimmt `generate-wf-config.py`, gesteuert über die Option `--defense tamaraw`. Der Bridge-Knoten wird der bestehenden Topologie hinzugefügt, und die Monitore werden so eingerichtet, dass sie ihren Tor-Verkehr nicht direkt zu einem Guard aus dem simulierten Netzwerk aufbauen, sondern zuerst über diesen Bridge-Knoten leiten.

Zusätzlich wird auf den Monitor-Hosts das Circuit-Padding deaktiviert. Damit wirkt in dieser Messreihe ausschliesslich Tamaraw auf den Zellfluss, und es überlagern sich keine zwei Verteidigungsverfahren. Die Tamaraw-Variante steht so in direkter Vergleichsposition zu den drei Circuit-Padding-Modi aus Tabelle 5.2.

¹⁰<https://github.com/websitefingerprinting/wfdef>

6 Validation und Evaluation

Die Resultate sind in Tabelle C.3 im Anhang vollständig aufgeführt.

6.1 Validierung des Versuchsaufbaus

Um den Versuchsaufbau zu validieren, werden drei Kennzahlen betrachtet. Erstens liegt der Anteil fehlgeschlagener Seitenabrufe in allen Closed-World-Läufen unter 5 % und ist innerhalb einer Klasse zwischen den Padding-Modi vergleichbar. Zweitens liegt die mittlere Paketanzahl pro Sample für die Circuit-Padding-Modi konsistent zwischen 580 und 640, was dafür spricht, dass die Seitenabrufe vergleichbar gross sind und sich nicht systematisch zwischen den Vergleichsgruppen unterscheiden, während Tamaraw die Sequenz erwartungsgemäss bis zur 5000-Werte-Grenze füllt. Drittens erreichen alle Modelle Klassifikationsraten deutlich über der Random-Basis (vergleiche Tabelle C.3), lernen also tatsächlich Merkmale des Verkehrs und reagieren nicht zufällig. Der Versuchsaufbau wird auf Basis dieser Kontrollen als valide angesehen.

6.2 Klassifikationsresultate

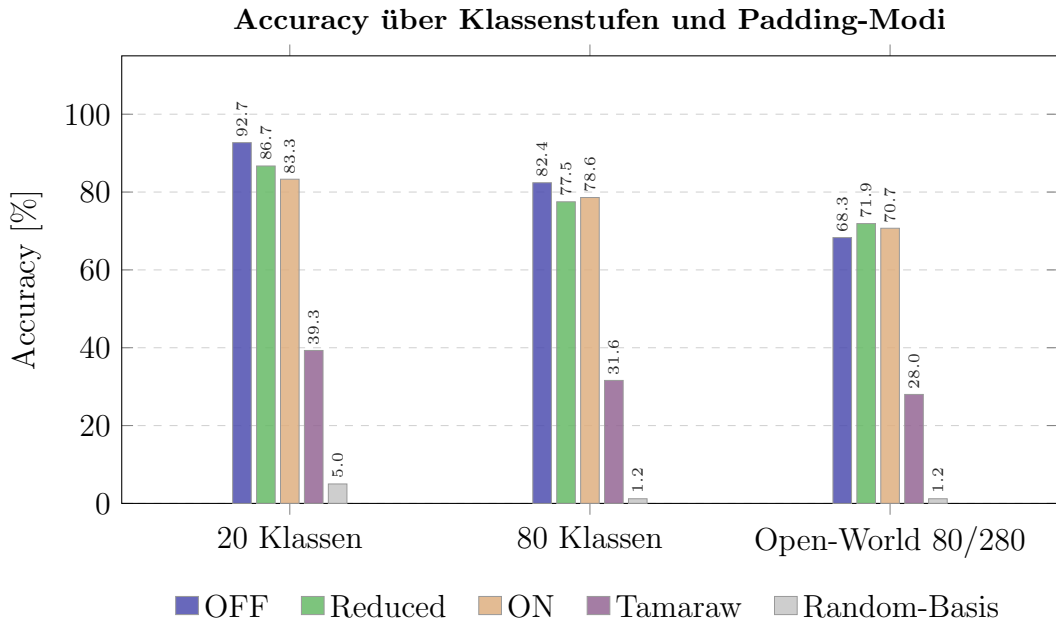


Abbildung 6.1: Accuracy des DF-Klassifikators für die vier untersuchten Verteilungen. Die genauen Accuracy- und F1-Werte sind in Tabelle C.3 im Anhang aufgeführt.

6.2.1 Skalierung mit der Klassenanzahl

Über alle Padding-Modi hinweg sinkt die Accuracy mit wachsender Klassenanzahl. Die ungeschützte Baseline fällt von 92.7 % bei 20 Klassen auf 82.4 % bei 80 Klassen und weiter auf 68.3 % im Open-World.

Mit jeder zusätzlichen Klasse muss der Klassifikator viele feinere Trennflächen statt weniger breiter Entscheidungsregionen lernen, was bei vergleichbarer Trainingsmenge pro Klasse die effektive Trennschärfe senkt.

Im Open-World tritt ein struktureller Effekt hinzu: Der Klassifikator entscheidet erst, ob ein Besuch zu einer überwachten Seite oder zur unüberwachten Sammelklasse gehört, und ordnet ihn im positiven Fall anschliessend einer konkreten überwachten Seite zu, was beide Stufen schwieriger macht. Diese Beobachtung ist konsistent mit Juarez et al., die für Open-World-Szenarien deutlich höhere Falsch-Positiv-Raten berichten und betonen, dass die in Closed-World-Studien gemeldeten Erkennungsraten in der Praxis nicht erreicht werden [31].

6.2.2 Effekt von Circuit Padding

Die Aktivierung des produktiv im Tor-Netzwerk eingesetzten Circuit Paddings (ON) senkt die Accuracy bei 20 Klassen von 92.7 % auf 83.3 %, was einer Reduktion um 9.4 Prozentpunkte entspricht. Bei 80 Klassen ist der Effekt deutlich kleiner: Die Accuracy sinkt um 3.8 Prozentpunkte von 82.4 % auf 78.6 %.

Eine mögliche Interpretation ist, dass das Klassifikationsproblem mit wachsender Klassenanzahl bereits ohne Padding fordernder wird, sodass das zusätzliche Rauschen durch Padding-Zellen relativ weniger ins Gewicht fällt.

Die Grössenordnung dieser Werte ist vergleichbar mit Sirinam et al., die für die ebenfalls auf Adaptive Padding basierende Verteidigung WTF-PAD in einem Closed-World-Setup mit 95 Klassen einen Rückgang des DF-Angriffs von 98.3 % auf 90.7 % berichten [2].

6.2.3 Reduced Padding und Open-World-Inversion

Der Reduced-Modus wird in der torrc-Konfiguration zusätzlich zu `CircuitPadding 1` mit `ReducedCircuitPadding 1` aktiviert und ist als sparsamere Variante des Circuit Paddings ausgelegt. Die intuitive Erwartung ist daher, dass seine Accuracy zwischen OFF und voll aktiviertem Padding (ON) liegt.

Bei 20 Klassen erfüllen die gemessenen 86.7 % diese Erwartung und liegen zwischen OFF und ON. Bei 80 Klassen kehrt sich die Reihenfolge um: Reduced erreicht 77.5 % und damit rund einen Prozentpunkt unter ON mit 78.6 %. Im Open-World-Szenario ist die Umkehr noch deutlicher, sowohl Reduced als auch ON liegen oberhalb der OFF-Baseline (68.3 %). Abbildung 6.2 ergänzt das Bild um Precision, Recall und F1 im Open-World-Setup und zeigt, dass die Unterschiede zwischen OFF, Reduced und ON auf allen drei Kennzahlen klein bleiben, während Tamaraw qualitativ abweicht.

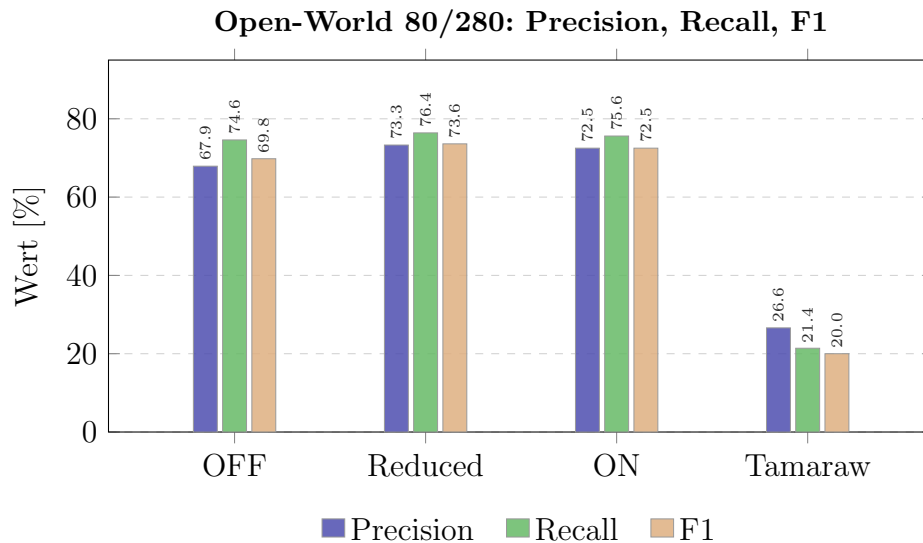


Abbildung 6.2: Precision, Recall und F1-Wert im Open-World-Setup (80 überwachte Seiten in einem Topf von 280). Eigene Darstellung aus `Experiment-Logs.csv`.

6.2.4 Mögliche Erklärungen der Inversion

Warum das Padding hier negativ wirkt, lässt sich aus dem vorliegenden Datenmaterial nicht eindeutig beantworten. Mehrere Effekte kommen als Ursache in Frage und müssten in weiteren Läufen gezielt überprüft werden.

Da pro Konfiguration nur ein einziger Shadow-Lauf mit festem Seed vorliegt, kann ein Teil der Differenz schlicht aus der Lauf-zu-Lauf-Streuung des Simulators stammen. Zufällige Anteile wie die Auswahl der Tor-Pfade oder die Streuung des Hintergrundverkehrs könnten zu Schwankungen führen.

Ebenso kommen Eigenschaften des Versuchsaufbaus selbst in Frage. Die Netzwerkgrösse von 0.01, die mit 150 Visits pro Klasse knappe Stichprobe und der strukturell homogene Simple-English-Wikipedia-Korpus engen den Merkmalsraum ein, was die Ergebnisse unberechenbarer machen könnte.

Welcher dieser Effekte tatsächlich zum Tragen kommt, lässt sich klären, wenn jede Konfiguration mit mehreren Seeds, einem grösseren Netzwerk und einem heterogeneren Korpus wiederholt wird (siehe Abschnitte 7.1.1, 7.1.2 und 7.1.3).

6.2.5 Vergleich mit Tamaraw

Tamaraw wurde als alternative Verteidigung in einer separaten Messreihe untersucht, um den Effekt mit dem des Circuit Paddings zu vergleichen.

Im Closed-World mit 20 Klassen sinkt die Accuracy unter Tamaraw auf 39.3 %, bei 80 Klassen auf 31.6 % und im Open-World auf 28.0 %. Damit fällt der Effekt deutlich stärker aus als beim Circuit Padding.

Auf Ebene der Konfusionsmatrix wird dieser qualitative Unterschied direkt sichtbar (Abbildung 6.3). Unter aktivem Circuit Padding bleibt die Diagonale gegenüber OFF weitgehend erhalten und die Fehlklassifikationen verteilen sich nur leicht breiter um die Hauptdiagonale. Unter Tamaraw löst sich die Diagonale dagegen weitgehend auf und die Vorhersagen verteilen sich breit über viele Klassen.

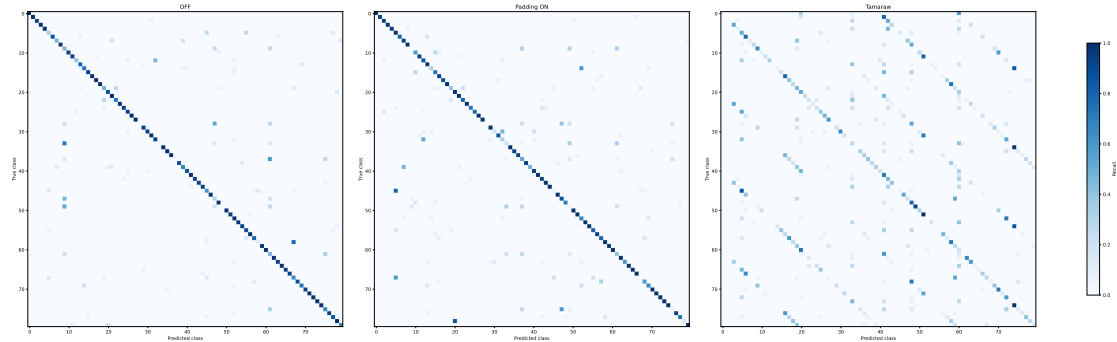


Abbildung 6.3: Konfusionsmatrizen im Closed-World-Setup mit 80 Klassen, normiert pro Zeile (Recall). Links die ungeschützte Baseline (OFF), Mitte mit aktivem Circuit Padding (ON), rechts mit Tamaraw. Eigene Darstellung.

6.2.6 Einordnung von Tamaraw

Trotz der starken Schutzwirkung bleibt Tamaraw deutlich über der Random-Basis von 5.0 % beziehungsweise 1.2 %. Die konstante Senderate alleine reicht also nicht aus, um den Verkehr vollständig in Rauschen aufzulösen.

Dennoch zeigt sich, dass Tamaraw die Erkennungsraten deutlich reduziert. Diese Beobachtung deckt sich mit den ursprünglich von Cai et al. für Tamaraw berichteten starken Schutzwirkungen [24] und mit der Einordnung von Circuit Padding als low-overhead-Verteidigung durch Kadianakis et al. [25]. Der zugehörige Preis für die deutlich stärkere Schutzwirkung von Tamaraw wird im folgenden Abschnitt 6.3 sichtbar.

6.3 Bandbreiten-Overhead und Kosten-Nutzen-Verhältnis

Der durch die Verteidigungen verursachte Bandbreiten-Overhead wird aus den Monitor-PCAPs der Closed-World-80-Läufe mit dem Skript `pcap_overhead.py` ermittelt. Pro Eintrag der `schedule.json` werden alle TCP-Pakete innerhalb des 30-Sekunden-Visit-Fensters auf der eth0-Schnittstelle des Monitors aufsummiert, getrennt nach Up- (Client → Netzwerk) und Down-Richtung. Für die Läufe `exp-baseline-80`, `exp-padding-80` und `exp-reduced-80` fließen je fünf Monitore in die Statistik ein (3000 Besuche pro Lauf), für `exp-tamaraw-80` aufgrund der etwa zwanzigfach grösseren PCAP-Dateien lediglich ein Monitor (600 Besuche).

Volles und reduziertes Circuit Padding erhöhen den mittleren Datenverkehr pro

Tabelle 6.1: Mittlerer Datenverkehr pro Besuch auf der Monitor-Schnittstelle und relativer Overhead gegenüber `exp-baseline-80`. Eigene Messung aus den Monitor-PCAPs mit `pcap_overhead.py`.

Lauf	Bytes Up	Bytes Down	Pakete Gesamt	Δ Bytes
exp-baseline-80	81 003	349 677	602	Basis
exp-padding-80	87 398	356 150	638	+3.0 %
exp-reduced-80	83 942	352 416	621	+1.3 %
exp-tamaraw-80	3 662 162	1 347 325	11 256	+1063.1 %

Besuch um lediglich 3.0 % beziehungsweise 1.3 % in Bytes und liegen damit innerhalb der Streubreite einzelner Besuche. Tamaraw-Padding vervielfacht den Datenverkehr hingegen um den Faktor 11.6 in Bytes und 18.7 in Paketen.

Die gemessenen Werte liegen deutlich über den von Cai et al. ursprünglich angegebenen rund 199 % [24]. Die eigene Messung erfasst jedoch nicht die protokollnahen Tor-Zellen, sondern TCP- und TLS-Pakete inklusive Framing auf der Monitor-Schnittstelle. Sie summiert zudem über das gesamte 30-Sekunden-Visit-Fenster, in dem Tamaraw auch nach abgeschlossenem Seitenabruf weiter Dummy-Zellen sendet, und beruht für Tamaraw nur auf einem einzelnen Monitor mit 600 Besuchen statt auf fünf Monitoren wie bei den übrigen Modi. Der absolute Wert von rund 1000 % ist daher nur eingeschränkt aussagekräftig und nicht direkt mit dem in der Forschungsliteratur berichteten Overhead vergleichbar, er belegt aber die Größenordnung der Mehrkosten von Tamaraw gegenüber dem Circuit Padding.

Im Verhältnis von Kosten zu Schutzwirkung steht Circuit Padding bei rund einem Prozentpunkt Accuracy-Reduktion pro Prozentpunkt Mehrverkehr im 20-Klassen-Setup, Tamaraw dagegen nur bei rund 0.05 Prozentpunkten. Tamaraw erkaufte seinen Schutz also mit einem überproportional hohen Bandbreitenaufschlag, der bereits in der ursprünglichen Auswertung von Cai et al. mit rund 199 % beziffert wird [24] und die Verteidigung für den realen Tor-Betrieb wenig praktikabel erscheinen lässt. Abbildung 6.4 visualisiert diesen Kosten-Schutz-Vergleich für die drei Verteidigungen im 80-Klassen-Setup.

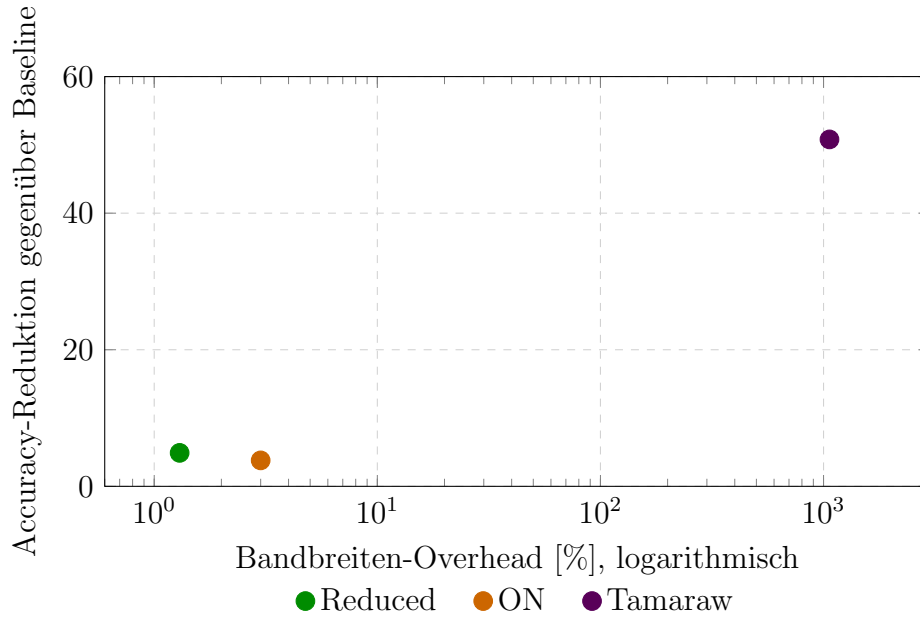


Abbildung 6.4: Bandbreiten-Overhead (logarithmisch) gegen Accuracy-Reduktion bei 80 Klassen. Reduced und ON liegen im unteren linken Bereich (geringe Kosten, geringer Schutz), Tamaraw rechts oben (hoher Schutz, hohe Kosten). Eigene Darstellung.

6.4 Ressourcen und Performance

Ergänzend zu den klassifikationsbezogenen Resultaten wird der Ressourcenverbrauch des Simulationsservers `shadowsrv-001` während einzelner Experimente protokolliert. Dazu wird das Skript `perfmon.py` verwendet, das im Intervall von fünf Sekunden Werte aus `/proc/stat`, `/proc/meminfo`, `statvfs` und `/proc/loadavg` ausliest und in CSV-Datei schreibt.

Betrachtet werden die drei zeitlich nacheinander ausgeführten Tamaraw-Läufe `exp-tamaraw-20v2`, `exp-tamaraw-80` und `exp-tamaraw-openworld`.

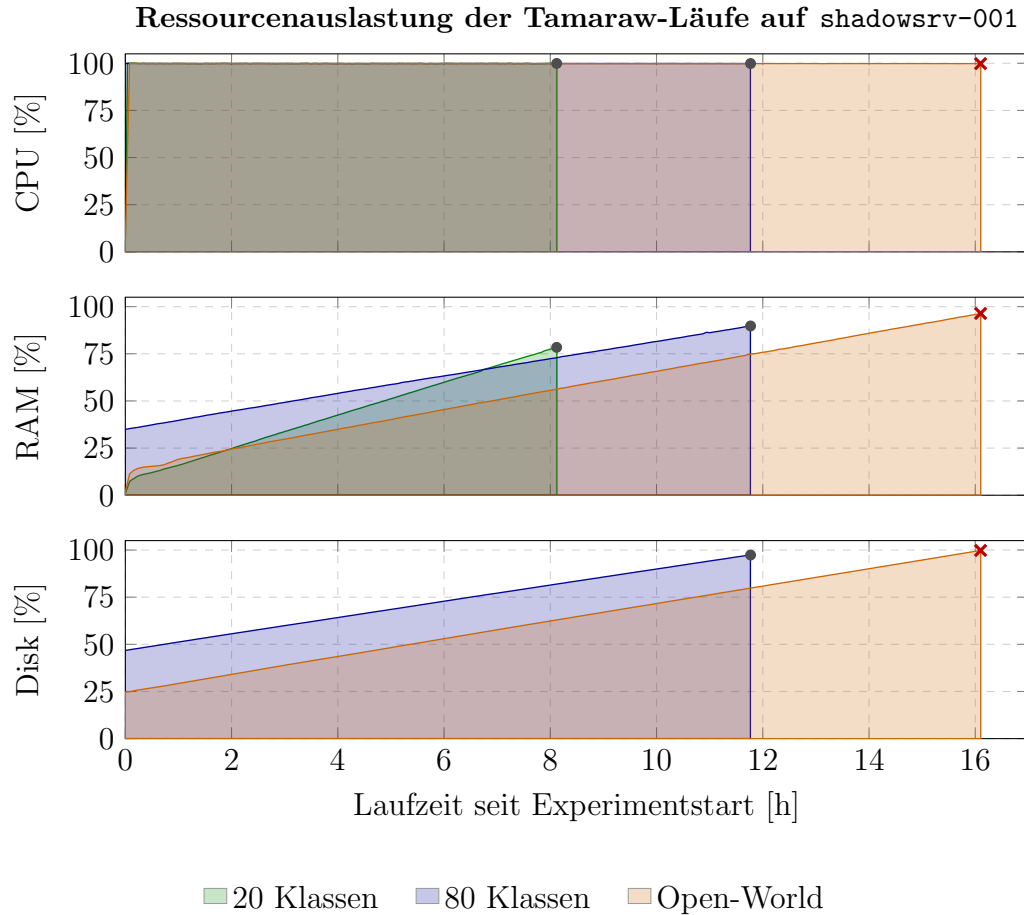


Abbildung 6.5: Auslastung von CPU, Arbeitsspeicher und Disk auf `shadowsrv-001` während der Tamaraw-Simulationen. Eigene Darstellung aus den `perfmon`- und `dstat`-Logs.

Über alle drei Läufe hinweg ist die CPU mit konstant rund 99.8 % hoch ausgelastet und bleibt am Anschlag. Arbeitsspeicher und Disk wachsen während der Laufzeit annähernd linear, da sowohl Shadow-Datenstrukturen als auch die PCAP-Aufzeichnungen monoton anwachsen.

Bei der grösseren Open-World-Konfiguration ist die Disk-Partition mit 156 GB die letztlich limitierende Ressource: `perfmon` und Shadow terminieren mit `ENOSPC`, bevor der Lauf regulär enden kann.

6.5 Einordnung und Limitationen

Die in diesem Kapitel berichteten Resultate sind als laborinterne Vergleichsmessungen zu lesen, nicht als Vorhersagen für die Wirkung der untersuchten Verteidigungen im Live-Tor-Netzwerk. Mehrere Faktoren begrenzen die Übertragbarkeit auf reale Bedingungen und werden im Folgenden offen benannt.

Schwerpunkt der Arbeit. Gemäss der in Anhang A dokumentierten Aufgabenstellung umfasst die vorliegende Arbeit sowohl den konstruktiven Aufbau einer Shadow-basierten WF-Pipeline als auch die experimentelle Auswertung mehrerer Verteidigungen. Der zeitliche Schwerpunkt liegt damit auf dem konstruktiven Teil, also dem Aufbau und der Validierung der Werkzeugketten sowie deren Integration mit WFLib. Für die eigentliche Experimentphase verblieb entsprechend nur limitiert Zeit, weshalb pro Konfiguration jeweils nur ein Shadow-Lauf mit festem Seed durchgeführt wurde und zusätzliche Variationen wie mehrere Seeds, grössere Netzwerkskalen oder ein heterogenerer Korpus nicht mehr untersucht werden konnten. Die entsprechenden Erweiterungen werden in Kapitel 7 als mögliche Folgearbeiten beschrieben.

Skalierung des simulierten Netzwerks. Die Experimente verwenden eine Netzwerkgrösse von 0.01, was rund 79 Relays und etwa 100 Hintergrund-Clients entspricht. Da das simulierte Netzwerk mit dieser Skalierung deutlich kleiner ist als das Live-Tor-Netzwerk, kann es sich anders oder weniger realistisch verhalten, und die absoluten Accuracy-Werte sind nicht unmittelbar auf das reale Netzwerk übertragbar.

Browser und Single-Tab-Modell. Anstelle eines vollwertigen Tor Browser wird ein gepatchter `wget2`-Client eingesetzt, der pro Besuch genau eine Seite mit allen referenzierten Ressourcen über Tor herunterlädt. `wget2` verarbeitet weder JavaScript noch dynamische DOM-Effekte und teilt Ressourcen-Requests anders auf parallele Verbindungen auf als ein realer Browser. Wang und Goldberg zeigen, dass bereits parallel offene Browser-Tabs die Erkennungsrate eines Closed-World-Klassifikators erheblich senken, da die resultierenden Verkehrsmuster Überlagerungen mehrerer Seiten darstellen [16]. Im realen Tor-Browser-Betrieb ist Multi-Tab-Verkehr die Regel und nicht die Ausnahme, weshalb die hier gemessenen Erkennungsraten den im Live-Netz erreichbaren Wert klar überschätzen dürften.

Statische Inhalte. Die Monitor-Clients beziehen ihre Inhalte aus einem statischen Snapshot der Simple English Wikipedia, ausgeliefert über den eigens gebauten `zimsrv.py`. Damit fehlen reale Dynamiken wie sich verändernde Werbeflächen, A/B-Tests, Drittanbieter-Tracker, CDN-Lastverteilung oder personalisierte Inhalte, die im Live-Web jede Seite über die Zeit verändern. Juarez et al. benennen genau diese Inhaltsveränderungen über die Zeit (concept drift) als einen Hauptgrund dafür, dass Closed-World-Resultate im realen Tor-Browser-Betrieb nicht reproduzierbar sind [31]. Im vorliegenden Versuchsaufbau ist concept drift ausgeschlossen, alle Besuche derselben Seite liefern bis auf Tor-bedingte Streuung dieselben Bytes.

Trainingsmenge, Inhaltskorpus und Eingangsgranularität. Pro Klasse fliesen in dieser Arbeit 150 Visits in die Trainings-, Validierungs- und Testaufteilung ein, was rund einem Siebtel der Stichprobe entspricht, mit der Sirinam et al. ihre 98.3% Baseline-Accuracy auf 95 Klassen erreichten [2]. Daneben unterscheidet sich auch der Inhaltskorpus: Sirinam et al. beobachten heterogene Live-

Webseiten aus den Top 100 von Alexa mit unterschiedlichen Frameworks, CDNs und Ressourcenprofilen, während im vorliegenden Aufbau ausschliesslich Artikel aus der Simple English Wikipedia über denselben `zimsrv` ausgeliefert werden, deren HTML-Gerüst und Ressourcenstruktur sich nur in den eigentlichen Inhalten unterscheiden. Hinzu kommt die in Abschnitt 3.10 beschriebene gröbere Granularität der Eingangsdaten. Jeder der 5000 Werte des Richtungsvektors entspricht in der vorliegenden Pipeline einem TCP-Paket, bei Sirinam et al. dagegen einer feineren Tor-Zelle [2]. Da eine Aufzeichnung im Median nur rund 600 Pakete umfasst (Tabelle C.4), besteht der Eingangsvektor überwiegend aus Nullpadding und trägt weniger feinkörnige Sequenzinformation als ein zellgenauer Trace. Diese Faktoren wirken in dieselbe Richtung und erschweren die Klassentrennung, sodass die im Vergleich zu Sirinam et al. tiefere Baseline-Accuracy (92.7 % bei 20 Klassen, 82.4 % bei 80 Klassen) plausibel wird.

Anzahl Wiederholungen. Pro Konfiguration wird genau ein Shadow-Lauf mit einem festen Seed durchgeführt. Differenzen in der Grössenordnung weniger Prozentpunkte sind daher mit Vorsicht zu interpretieren. Zusätzlich liegen pro Klasse nur 150 Visits vor, wovon bei einem 10 %-Test-Split lediglich rund 15 Samples in die Accuracy einfließen, sodass einzelne Fehlklassifikationen die berichteten Werte merkbar bewegen können.

Insgesamt zeigt die Auswertung, dass das produktiv ausgelieferte Circuit Padding im untersuchten Laborkontext einen messbaren, aber kleinen Beitrag zur Verteidigung gegen DF-Angriffe leistet (zwischen 3.8 und 9.4 Prozentpunkte Accuracy-Reduktion im Closed-World), bei einem praktisch vernachlässigbaren Bandbreitenaufschlag von unter 5 %. Tamaraw erreicht in derselben Umgebung eine um eine Grössenordnung stärkere Schutzwirkung, erkaufte sich diese aber mit einem erheblichen Bandbreiten-Overhead. Cai et al. beziffern diesen in ihrer ursprünglichen Auswertung mit rund 199 % [24], und auch die eigene, methodisch gröbere PCAP-Messung weist ihn als Vielfaches des Circuit-Padding-Aufwands aus. Tamaraw ist damit in seiner heutigen Form nicht für den breiten Einsatz im Tor-Netzwerk geeignet. Die genannten Limitationen, insbesondere das Single-Tab-Modell und die kleine Skalierung, legen nahe, dass beide Verteidigungen unter realistischeren Bedingungen tendenziell stärker wirken, da der Klassifikator bereits in der Baseline mit höherem Rauschen rechnen müsste.

6.6 Erreichung der Ziele und Forschungsfragen

Abschliessend werden die in der Aufgabenstellung (Anhang A) vereinbarten Ziele und die in Kapitel 1 formulierten Forschungsfragen den erzielten Resultaten gegenübergestellt.

Die drei in Kapitel 1 angestrebten Resultate liegen vor. Eine Shadow-Simulationsumgebung wurde auf einer Virtuellen Maschine aufgebaut und über die in Kapitel 5 beschriebene Werkzeugkette steuerbar gemacht. In dieser Umgebung wurde ein WF-Datenset erzeugt, und der DF-Angriff wurde mit WFLib über alle untersuchten

Konfigurationen durchgeführt.

Auch die drei Forschungsfragen lassen sich im Rahmen des Laboraufbaus beantworten. Die erste Frage nach der Veränderung der Erkennungsrate unter verschiedenen Circuit-Padding-Einstellungen wird in den Abschnitten 6.2.2 und 6.2.3 beantwortet, wonach produktiv ausgeliefertes Circuit Padding die Accuracy im Closed-World um 3.8 bis 9.4 Prozentpunkte senkt. Die zweite Frage nach den dokumentierten Limitationen simulierter WF-Angriffe und der Einordnung der eigenen Ergebnisse wird in Abschnitt 6.5 behandelt. Die dritte, im Verlauf der Arbeit ergänzte Frage nach der zusätzlichen Schutzwirkung von Tamaraw wird in den Abschnitten 6.2.5 und 6.3 beantwortet, wonach Tamaraw eine um eine Größenordnung stärkere, aber mit einem hohen Bandbreiten-Overhead von rund 199 % [24] erkaufte Schutzwirkung erzielt.

Die ursprüngliche Zielsetzung, eine Shadow-basierte WF-Simulation auf moderater Hardware mit einer unmodifizierten Tor-Version aufzubauen und den Effekt von Circuit Padding darin zu beobachten, wird damit erfüllt.

7 Ausblick

Die in Kapitel 6 berichteten Resultate beantworten die in Kapitel 1 gestellten Forschungsfragen im Rahmen des gewählten Laboraufbaus. Sie lassen jedoch eine Reihe von Fragen offen, die innerhalb der verfügbaren Bearbeitungszeit nicht mehr abgedeckt werden konnten. Dieses Kapitel ordnet die wichtigsten Folgearbeiten in zwei Gruppen. Abschnitt 7.1 beschreibt Erweiterungen, die unmittelbar auf dem aufgebauten WF-Versuch aufsetzen, Abschnitt 7.2 skizziert alternative Untersuchungsszenarien, die das Bedrohungsmodell wechseln oder die Infrastruktur für andere Fragestellungen nutzen. Abschnitt 7.4 schliesst mit einer Reflexion über die erreichten Ziele und den Nutzen der aufgebauten Werkzeugkette für künftige Arbeiten.

7.1 Weiterentwicklung der Website-Fingerprinting-Experimente

Die folgenden Erweiterungen bauen direkt auf dem in dieser Arbeit aufgebauten Versuch auf und liessen sich mit der bestehenden Pipeline ohne grundlegende Umbauten umsetzen. Sie adressieren primär die in Kapitel 6 benannten methodischen Limitationen und zielen darauf, die gemessene Schutzwirkung von Circuit Padding unter realistischeren Bedingungen abzusichern.

7.1.1 Grössere Datenbasis: mehrere Seeds und mehr Visits

Jede in dieser Arbeit berichtete Kennzahl stammt aus genau einem Shadow-Lauf mit festem Seed. Damit lassen sich Differenzen in der Grössenordnung weniger Prozentpunkte, wie jene zwischen ON und Reduced bei 80 Klassen oder die Inversion der Padding-Wirkung im Open-World-Setup, nicht von der Streuung des Simulators unterscheiden. Eine Wiederholung jedes Hauptexperiments mit mehreren Seeds würde Mittelwerte, Standardabweichungen und Konfidenzintervalle pro Konfiguration ermöglichen. Ergänzend liessen sich die in Abschnitt 6.5 als knapp benannten 150 Visits pro Klasse erhöhen, da bei einem 10 %-Test-Split nur rund 15 Samples in die Accuracy einfliessen und einzelne Fehlklassifikationen die Werte merkbar bewegen können.

7.1.2 Realistischere Inhalte

Die in dieser Arbeit verwendeten Klassen stammen aus einem Zufalls-Sample der Simple English Wikipedia und sind strukturell sehr homogen, da Markup, Pfadschema und Ressourcenprofil pro Artikel weitgehend identisch sind. Zudem schliesst das statische ZIM-Snapshot-Modell jede zeitliche Drift der Inhalte aus, was Juarez et al. als einen der zentralen Gründe nennen, weshalb Closed-World-Resultate aus Laborbedingungen im Live-Web nicht reproduzierbar sind [31]. Bei-

des erhöht die Reproduzierbarkeit, vergrößert aber den Abstand zu produktiven Verhältnissen und schränkt den direkten Vergleich mit Forschungsliteratur-Resultaten ein, die wie Sirinam et al. auf heterogenen Live-Korpora wie Alexa Top 100 beruhen [2]. Eine Folgearbeit könnte statt der Wikipedia andere Websites in den Shadow-Container spiegeln, etwa das rund 100 Seiten umfassende Set von Jansen und Wails [4] oder einen statisch gespiegelten Auszug einer Alexa-Top-N-Liste. Da Shadow-Knoten keinen externen Netzzugang besitzen, ist dafür ein vorgelagerter Spiegel- oder Caching-Schritt nötig, der die Inhalte als eingefrorene Momentaufnahme bereitstellt. Ergänzend liesse sich diese Momentaufnahme kontrolliert variieren, etwa durch veränderte Ressourcenreihenfolge oder eine künstliche Aktualisierung zwischen Trainings- und Testfenster, um die Auswirkung zeitlicher Drift auf die Padding-Wirkung reproduzierbar zu untersuchen.

7.1.3 Grössere Netzwerk-Skalierung und Speicher

Die Netzwerkgrösse von 0.01 entspricht rund einem Hundertstel des Live-Tor-Netzwerks. Eine Skalierung auf 0.05 oder 0.1 würde die Anzahl Relays in den Bereich mehrerer Hundert bis Tausend bringen und damit dem realen Netzwerk näher kommen. Voraussetzung ist die in Abschnitt 6.4 angeregte Umstellung der Verkehrsaufzeichnung auf Cell-Trace-Logs, ohne die der Speicherbedarf bereits bei der derzeitigen Skalierung an die 156-GB-Grenze stösst. Wird am PCAP-Verfahren festgehalten, wäre für künftige Läufe eine deutlich grössere Disk von Vorteil, da die Paketmitschnitte den Speicher schneller füllen als alle übrigen Artefakte zusammen. Mit beiden Massnahmen liesse sich erstmals untersuchen, ob die hier gemessene Schutzwirkung von Circuit Padding mit wachsender Netzwerkgrösse abnimmt, gleich bleibt oder sich invertiert.

7.1.4 Weitere Verteidigungen

Die Arbeit vergleicht Circuit Padding gegen die historisch bedeutsame Tamaraw-Verteidigung. Seit deren Veröffentlichung sind weitere Schutzansätze publiziert worden, die einen günstigeren Kosten-Schutz-Punkt im in Abschnitt 6.3 aufgespannten Trade-off versprechen, insbesondere RegulaTor mit lastabhängiger Senderate und FRONT mit zufallsbasierter Padding-Verteilung am Sitzungsanfang. Eine Folgearbeit könnte die aufgebaute Pipeline ohne grosse Änderungen auf diese Verteidigungen anwenden, sofern eine Referenzimplementierung als Tor-Patch oder als wfdefproxy-Plugin vorliegt [30], und so den in Tabelle 6.1 aufgespannten Vergleich erweitern.

7.1.5 Übergang ins Live-Tor-Netzwerk

Alle berichteten Resultate stammen aus dem Shadow-Simulator und sind daher reproduzierbar, aber auf den simulierten Kontext beschränkt. Eine perspektivische Erweiterung wäre die Übertragung des Versuchsaufbaus auf das Live-Tor-Netzwerk unter Beibehaltung der bestehenden Pipeline, wobei Monitor-Client und Verkehrsaufzeichnung in einer abgeschirmten Umgebung mit eigenem Tor-Daemon betrie-

ben würden und sämtliche Inhalte aus einer eigens kontrollierten Quelle stammen müssten. So liesse sich messen, in welchem Umfang die in Abschnitt 6.5 benannten Limitationen die Schutzwirkung von Circuit Padding über- oder unterschätzen, ohne Verkehr realer Nutzender einzubeziehen. Aus ethischer Sicht ist eine solche Erweiterung sorgfältig zu planen, da bereits passive Messungen im Live-Netzwerk die Anonymität anderer Beteiligter berühren können.

7.2 Alternative Untersuchungsszenarien

Über die unmittelbare Fortführung des WF-Versuchs hinaus eröffnet die aufgebaute Shadow-Umgebung weitere Untersuchungsrichtungen. Diese wechseln entweder das Bedrohungsmodell oder nutzen die Infrastruktur für verwandte Fragestellungen, die im Rahmen dieser Arbeit nicht mehr bearbeitet werden konnten.

7.2.1 Simulation von Korrelationsangriffen

Die aus Sicht des Autors wertvollste Erweiterung wäre, die aufgebaute Simulationsumgebung so weiterzuentwickeln, dass sich Korrelationsangriffe untersuchen lassen. Während WF einen lokalen Beobachter am Eingang des Schaltkreises voraussetzt, modelliert ein Korrelationsangriff einen Gegner, der die Verkehrszeitreihen am Eingang und am Ausgang vergleicht und so Sender und Empfänger einander zuordnet [10, 9]. Dieses Bedrohungsmodell ist besonders praxisrelevant, da entsprechende Angriffe gegen Tor nicht nur theoretisch sind, sondern in der realen Welt bereits mehrfach beobachtet wurden [14]. Shadow bietet dafür eine günstige Ausgangslage, weil der Verkehr an beiden Enden eines Schaltkreises in einer kontrollierten, reproduzierbaren Umgebung aufgezeichnet werden kann, ohne den Verkehr realer Nutzender einzubeziehen. Eine solche Erweiterung würde es erlauben, das alternative Bedrohungsmodell des globalen passiven Beobachters quantitativ einzuordnen und der Wirkung der untersuchten Padding-Modi gegenüberzustellen, da Circuit Padding und Tamaraw primär auf andere Merkmale zielen als zeitliche Korrelation.

7.2.2 Erkennung von Tor-Verkehr

Eng verwandt mit WF, aber mit eigenständigem praktischem Wert, ist die Frage, ob sich Tor-Verkehr mit einem fingerprinting-ähnlichen Verfahren überhaupt von gewöhnlichem Verkehr unterscheiden lässt [32]. Diese Unterscheidung ist die Grundlage staatlicher Zensur, etwa durch die Great Firewall in China, und der Grund, weshalb Tor pluggable transports wie obfs4 einsetzt, die den Verkehr verschleiern sollen [18]. Eine Folgearbeit könnte im Shadow-Aufbau sowohl Tor- als auch Nicht-Tor-Verkehr erzeugen und prüfen, wie zuverlässig ein Klassifikator beide trennt und wie stark obfs4 diese Trennung erschwert. Der praktische Nutzen liegt höher als beim reinen WF, da Verkehrserkennung der erste Schritt jeder Blockade ist und damit unmittelbar die Erreichbarkeit von Tor in zensierten Netzen betrifft.

7.3 Zusammenfassung der offenen Punkte

Die genannten Folgearbeiten lassen sich nach Aufwand und erwartetem Erkenntnisgewinn ordnen. Mit überschaubarem Aufwand und hohem Erkenntnisgewinn lässt sich die Datenbasis durch mehrere Seeds und mehr Visits verbreitern, was die in Kapitel 6 benannten methodischen Lücken unmittelbar adressiert. Den grössten Erkenntnisgewinn versprechen die Erweiterung der Umgebung um Korrelationsangriffe und die Erkennung von Tor-Verkehr, da beide ein praxisrelevantes und real bereits beobachtetes Bedrohungsmodell adressieren. Die übrigen Punkte erfordern strukturelle Erweiterungen, eröffnen aber einen Pfad, auf dem die aufgebaute Infrastruktur schrittweise an realistischere Bedingungen herangeführt werden kann.

7.4 Reflexion

Rückblickend bestätigt sich die in Abschnitt 6.5 getroffene Einordnung, wonach der zeitliche Schwerpunkt der Arbeit auf dem konstruktiven Teil lag. Der Aufbau einer reproduzierbaren WF-Pipeline im Shadow-Simulator erwies sich als deutlich aufwändiger als zu Projektbeginn angenommen. Mehrere Komponenten mussten erst aufeinander abgestimmt werden, bevor ein vollständiger Lauf von der Konfiguration bis zum trainierten Klassifikator gelang, darunter die Generierung der Shadow-Konfiguration, ein eigener ZIM-basierter Webserver als Ersatz für nicht kompatible Standardlösungen und die Aufbereitung der PCAPs in das von WFLib erwartete Format.

Als bleibendes Ergebnis dieses Aufwands steht eine Sammlung von Python-Werkzeugen zur Verfügung, die den gesamten Ablauf von der Simulation bis zur Auswertung kapselt und über `shadowctl.py` mit wenigen Parametern steuerbar macht. Damit lässt sich eine WF-Simulation mit unterschiedlicher Klassenanzahl, Besuchsanzahl und Padding-Konfiguration künftig mit erheblich geringerem manuellem Aufwand wiederholen, als es für die vorliegende Arbeit nötig war. Diese Werkzeugkette versteht der Autor als eigenständigen Beitrag der Arbeit, da sie die Einstiegshürde für vergleichbare Experimente senkt und als Grundlage für die in den vorangehenden Abschnitten skizzierten Folgearbeiten dient.

Die in der Aufgabenstellung (Anhang A) vereinbarten Ziele und die daraus abgeleiteten Forschungsfragen wurden im Rahmen des gewählten Laboraufbaus erreicht, wie in Abschnitt 6.6 im Einzelnen dargelegt wird. Auf dieser Grundlage können künftige Arbeiten unmittelbar aufbauen und den Versuch entlang der genannten Richtungen vertiefen, ohne die Infrastruktur erneut von Grund auf aufbauen zu müssen.

Literaturverzeichnis

- [1] R. Jansen, J. Tracey, and I. Goldberg, “Once is never enough: Foundations for sound statistical inference in Tor network experimentation,” in *30th USENIX Security Symposium (Sec)*, 2021. See also <https://neverenough-sec2021.github.io>.
- [2] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pp. 1928–1943, 2018.
- [3] R. Jansen and N. Hopper, “Shadow: Running tor in a box for accurate and efficient experimentation,” tech. rep., 2011.
- [4] R. Jansen and R. Wails, “Data-explainable website fingerprinting with network simulation,” *Proceedings on Privacy Enhancing Technologies*, 2023.
- [5] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The second-generation onion router,” 2004.
- [6] A. Mani, T. Wilson-Brown, R. Jansen, A. Johnson, and M. Sherr, “Understanding tor usage with privacy-preserving measurement,” in *Proceedings of the Internet Measurement Conference 2018*, pp. 175–187, 2018.
- [7] The Tor Project, “A short introduction to tor.” <https://spec.torproject.org/intro/index.html>, 2026. [Accessed 2026-05-23].
- [8] The Tor Project, “Tor directory protocol, version 3.” <https://spec.torproject.org/dir-spec/>, 2026. [Accessed 2026-05-03].
- [9] M. Nasr, A. Bahramali, and A. Houmansadr, “Deepcorr: Strong flow correlation attacks on tor using deep learning,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pp. 1962–1976, 2018.
- [10] S. J. Murdoch and G. Danezis, “Low-cost traffic analysis of Tor,” in *2005 IEEE Symposium on Security and Privacy (S&P’05)*, pp. 183–195, IEEE, 2005.
- [11] Government Communications Headquarters, “Tor Stinks.” Slide deck, veröffentlicht durch The Guardian, Oktober 2013, <https://www.theguardian.com/world/interactive/2013/oct/04/tor-stinks-nsa-presentation-document>, 2012. [Accessed 2026-05-28].
- [12] The Tor Project, “Hidden services, current events, and Freedom Hosting.” <https://blog.torproject.org/hidden-services-current-events-and-freedom-hosting/>, 8 2013. [Accessed 2026-05-28].
- [13] Electronic Frontier Foundation, “The playpen cases: Frequently asked questions.” <https://www.eff.org/pages/playpen-cases-frequently-asked-questions>, 2017. [Accessed 2026-05-28].
- [14] R. Dingledine, “Tor security advisory: “relay early” traffic confirmation attack.” <https://blog.torproject.org/tor-security-advisory-relay-early-traffic-confirmation-attack/>, 7 2014. [Accessed 2026-05-28].
- [15] nusenu, “Is “KAX17” performing de-anonymization attacks against Tor users?.” <https://nusenu.medium.com/is-kax17-performing-de-anonymization-attacks-against-tor-users-42e566defce8>, 12 2021. [Accessed 2026-05-28].
- [16] T. Wang and I. Goldberg, “On realistically attacking tor with website fingerprinting,” *Proceedings on Privacy Enhancing Technologies*, 2016.

- [17] G. Cherubin, R. Jansen, and C. Troncoso, “Online website fingerprinting: Evaluating website fingerprinting attacks on tor in the real world,” in *31st USENIX Security Symposium (USENIX Security 22)*, pp. 753–770, 2022.
- [18] P. Winter and S. Lindskog, “How the Great Firewall of China is blocking Tor,” in *2nd USENIX Workshop on Free and Open Communications on the Internet (FOCI 12)*, 2012.
- [19] R. Jansen, J. Newsome, and R. Wails, “Co-opting linux processes for high-performance network simulation,” in *USENIX Annual Technical Conference*, 2022. See also <https://netsim-atc2022.github.io>.
- [20] The Tor Project, “Tor Metrics — About.” <https://metrics.torproject.org/about.html>, 2026. [Accessed 2026-05-03].
- [21] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, “Website fingerprinting at internet scale,” in *NDSS*, vol. 1, p. 23477, 2016.
- [22] J. Hayes and G. Danezis, “k-fingerprinting: A robust scalable website fingerprinting technique,” in *25th USENIX Security Symposium (USENIX Security 16)*, pp. 1187–1203, 2016.
- [23] M. S. Rahman, P. Sirinam, N. Mathews, K. G. Gangadhara, and M. Wright, “Tik-tok: The utility of packet timing in website fingerprinting attacks,” *arXiv preprint arXiv:1902.06421*, 2019.
- [24] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, “A systematic approach to developing and evaluating website fingerprinting defenses,” in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pp. 227–238, 2014.
- [25] G. Kadianakis, T. Polyzos, M. Perry, and K. Chatzikokolakis, “Tor circuit fingerprinting defenses using adaptive padding,” *arXiv preprint arXiv:2103.03831*, 2021.
- [26] M. Juárez, M. Imani, M. Perry, C. Diaz, and M. Wright, “Wtf-pad: toward an efficient website fingerprinting defense for tor,” *CoRR*, abs/1512.00524, 2015.
- [27] A. Kwon, M. AlSabah, D. Lazar, M. Dacier, and S. Devadas, “Circuit fingerprinting attacks: Passive deanonymization of Tor hidden services,” in *24th USENIX Security Symposium (USENIX Security 15)*, pp. 287–302, 2015.
- [28] H. Balzert, M. Schröder, and C. Schäfer, *Wissenschaftliches Arbeiten - Ethik, Inhalt & Form wiss. Arbeiten, Handwerkszeug, Quellen, Projektmanagement, Präsentation, 3. Auflage*. 2022.
- [29] The Tor Project Network Health Team, “OnionPerf — A performance measurement tool for the Tor network.” <https://gitlab.torproject.org/tpo/network-health/onionperf>, 2026. [Accessed 2026-05-03].
- [30] “WFDefProxy: Pluggable transport implementation of website fingerprinting defenses (tamaraw, front, regulator, surakav).” <https://github.com/websitefingerprinting/wfdef>. Branch v2, accessed 2026-06-07.
- [31] M. Juarez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt, “A critical evaluation of website fingerprinting attacks,” in *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, pp. 263–274, 2014.
- [32] F. A. Saputra, I. U. Nadhori, and B. F. Barry, “Detecting and blocking onion router traffic using deep packet inspection,” in *2016 International Electronics Symposium (IES)*, pp. 283–288, IEEE, 2016.
- [33] X. Deng, Q. Yin, Z. Liu, X. Zhao, Q. Li, M. Xu, K. Xu, and J. Wu, “Robust multi-tab website fingerprinting attacks in the wild,” in *2023 IEEE symposium on security and privacy (SP)*, pp. 1005–1022, IEEE, 2023.

A Aufgabenstellung

Dieser Anhang enthält die von der auftraggebenden Stelle unterzeichnete Aufgabenstellung zur vorliegenden Arbeit im Original. Die folgenden Seiten sind eine unveränderte Wiedergabe des Dokuments, das im Vorfeld der Arbeit mit der HSLU als auftraggebender Stelle vereinbart wurde.

Aufgabenstellung

Modul:	Dept I BAA FS26
Titel:	Praktische De-Anonymisierung im Tor-Netzwerk: Einfluss von Circuit Padding auf Website-Fingerprinting im Shadow Netzwerk Simulator
Ausgangslage und Problemstellung:	Mit dem Tor Netzwerk kann der eigene Internetzugang anonymisiert werden. Verschiedene Gegenmassnahmen wurden entwickelt um diese Anonymisierung wieder rückgängig zu machen. Mit Website Fingerprinting (WF) werden Muster von Websites aufgezeichnet, wie die Anzahl der ein und ausgehenden Paketen. Damit können Rückschlüsse auf die aufgerufenen Webseiten gezogen werden. WF Angriffe können auf einem kompromittierten Tor Entry Node oder einem passiven Beobachter wie dem ISP ausgeführt werden (3). 2018 wurde als Gegenmassnahme Circuit Padding eingeführt, welches adaptive Dummy-Pakete zwischen Client und Middle Node einfügt. Damit wird das Traffic Muster gegenüber WF verschleiert. Im Tor-Client kann das circuit padding konfiguriert werden und ist standardmässig aktiviert. Mit dem 2011 vorgestellten (1) Shadow Netzwerksimulator können Tor-Netzwerke lokal und reproduzierbar simuliert werden. Website Fingerprinting Attacks können damit zuverlässig simuliert werden (2). 1: https://apps.dtic.mil/sti/tr/pdf/ADA559181.pdf 2: https://petsymposium.org/popets/2023/popets-2023-0125.pdf 3: https://petsymposium.org/popets/2016/popets-2016-0027.pdf
Ziel der Arbeit und erwartete Resultate:	- Wie verändert sich die Erkennungsrate der DF Angriffe mit unterschiedlichen Circuit Padding Settings? - Welche bekannten Limitationen simulierter und laborbasierter WF Angriffe sind in der Literatur dokumentiert, und wie sind die experimentellen Ergebnisse dieser Arbeit einzuordnen? - Aufgebaute Shadow Simulationsumgebung in einer VM von labservices.ch - Ein Website Fingerprinting Dataset wurde erstellt - Deep Fingerprinting (DF) WF Angriff durchgeführt mit dem Tool WFLib DF: https://github.com/deep-fingerprinting/df WFLib: https://github.com/Xinhao-Deng/Website-Fingerprinting-Library
Gewünschte Methoden, Vorgehen:	- Literatur und Technologieanalyse (Aufarbeitung aktueller Stand der Technik) - Experiment. Versuchsaufbau einer Tor Network Simulation im Shadow Tool. Durchführung von Website Fingerprinting Angriffen.
Kreativität, Methoden, Innovation:	Den Einfluss von Circuit Padding auf Website Fingerprinting Angriffe unter reproduzierbaren lokalen Bedingungen testen können.

Sonstige Bemerkungen:	

Projektteam

Student:in 1:	Oliver Staub
Betreuer:in:	Eskhita

Auftraggeber

Firma:	HSLU
Ansprechperson:	Dr. Radwan Eskhita
Funktion:	Dozent
Strasse:	
PLZ/Ort:	
Telefon:	0796657472
E-Mail:	radwan.eskhita@hslu.ch
Website:	

B Serverumgebung

Dieser Anhang dokumentiert die für die Experimente in Kapitel 5 verwendete Serverumgebung in der Form, wie sie zum Zeitpunkt der Inbetriebnahme bereitgestellt und im Verlauf der Arbeit eingerichtet wurde. Ziel ist die nachvollziehbare Beschreibung der konkreten Hardware- und Software-Konfiguration, sodass die in Kapitel 6 dargestellten Resultate auf einer vergleichbaren Maschine reproduziert werden können.

B.1 Bereitstellung

Die virtuelle Maschine wurde von HSLU Lab Services im Rahmen der Bachelorarbeit bereitgestellt. Die wesentlichen Eckdaten der Bereitstellung sind in Tabelle B.1 zusammengefasst.

Tabelle B.1: Eckdaten der bereitgestellten virtuellen Maschine.

Attribut	Wert
Bereitsteller	HSLU Lab Services
Hostname	<code>shadowsrv-001.prod.projects.ls.eee.intern</code>
Zugriff	SSH, ausschliesslich aus dem HSLU-Netzwerk (Campus-WLAN oder VPN)
Ausserbetriebnahme	1. August 2026

Beim ersten Login auf der VM wurde das initiale Passwort des Administratorkontos in ein neues, nicht weitergegebenes Passwort geändert. Die initialen Zugangsdaten werden in dieser Arbeit aus naheliegenden Sicherheitsgründen nicht wiedergegeben.

Die VM teilt sich gemäss Auskunft von Lab Services ein Subnetz mit weiteren projektgebundenen virtuellen Maschinen anderer Studierender. Aus diesem Grund wurde der SSH-Dienst auf Public-Key-Authentifizierung umgestellt und das Passwort-Login nach dem ersten Login deaktiviert. Über die VM hinaus sind keine weiteren Dienste nach aussen exponiert.

B.2 Hardware

Tabelle B.2 zeigt die zugesagten Hardware-Ressourcen. Diese Werte gelten für die VM in ihrer Gesamtheit, also inklusive Betriebssystem und Hilfsdienste.

Tabelle B.2: Hardware-Ausstattung der virtuellen Maschine.

Komponente	Wert
vCPU	32 Kerne
Arbeitsspeicher	128 GB
Speicher	156 GB (LVM, <code>ubuntu-vg/ubuntu-lv</code>)

Die Belegung des Root-Dateisystems mit zunächst 33 GB beinhaltet das Basis-Ubuntu-System sowie die in Abschnitt B.3 aufgelisteten Software-Komponenten. Während eines Simulationslau-

fes wächst die Belegung in erster Linie durch die PCAP-Aufzeichnungen. Bei voller Aufzeichnung aller Eingangs- und Ausgangsknoten wären die 123 GB freier Speicher schnell erschöpft.

B.3 Software

Auf der VM ist das Betriebssystem Ubuntu in einer aktuellen Long-Term-Support-Variante installiert. Tabelle B.3 listet die für die Experimente relevanten Software-Komponenten und ihre Quellen.

Tabelle B.3: Software-Komponenten auf der virtuellen Maschine.

Komponente	Quelle	Verwendung
Ubuntu Server	Bereitstellung Lab Services	Basisbetriebssystem
Shadow	GitHub <code>shadow/shadow</code>	Netzwerkssimulator
Tor	Standardpaket / Tor-Projekt	Tor-Daemon in Shadow
Tornettools	GitHub <code>shadow/tornettools</code>	Generierung der Shadow-Konfiguration
wget2 (gepatcht)	GitLab + Patch aus <code>explainwf-popets2023</code>	Webclient mit SOCKS-Unterstützung
libzim (Python)	PyPI	Lesen der ZIM-Datei
zimsrv.py	Eigene Entwicklung	Webserver in der Simulation
Python (Toolsenv)	Bereitstellung über <code>toolsenv</code>	Laufzeit für die Hilfsskripte

Die einmalige Einrichtung der genannten Komponenten erfolgt über das Skript `setup-wf-server.sh`, das die Abhängigkeiten installiert, `wget2` mit dem SOCKS-Patch baut und die Hilfsskripte (`newnym.py`, `zimsrv.py`, `generate-urls.py`) auf die VM kopiert.

B.4 Datenablage und Verzeichnisstruktur

Innerhalb des Heimverzeichnisses des Administratorkontos wird die in Tabelle B.4 dargestellte Verzeichnisstruktur verwendet. Die genauen Pfade sind in den Hilfsskripten als Konstanten hinterlegt und können bei Bedarf angepasst werden.

Tabelle B.4: Verzeichnisstruktur auf der VM.

Pfad	Inhalt
<code>~/tornettools</code>	Installation und Arbeitsverzeichnis von Tornettools
<code>~/toolsenv</code>	Python-Virtualenv für Tornettools, libzim und Hilfsskripte
<code>~/wikidata</code>	ZIM-Datei der Simple-English-Wikipedia
<code>~/wget2_noinstall</code>	Mit SOCKS-Patch gebautes <code>wget2</code>
<code>~/exp-*</code>	Pro Experiment ein Unterverzeichnis mit <code>shadow.config.yaml</code> , <code>schedule.json</code> , Logs und Pcaps

Beim Pull der Resultate auf die lokale Arbeitsstation wird pro Experiment dasselbe Unterverzeichnisschema gespiegelt, sodass lokal und auf der VM dieselbe Struktur vorliegt. Damit ist die manuelle Nachverfolgung von Resultaten zu ihren ursprünglichen Konfigurationen ohne zusätzliche Werkzeuge möglich.

B.5 Ressourcenauslastung der Tamaraw-Läufe

Die in Abschnitt 6.4 gezeigte Abbildung 6.5 fasst den Ressourcenverlauf der drei Tamaraw-Läufe grafisch zusammen. Tabellen B.5 bis B.7 dokumentieren die zugrunde liegenden Stichproben im Zwei-Stunden-Raster sowie den jeweils letzten von `perfmon.py` geschriebenen Wert. Die Werte stammen aus den CSV-Dateien `perfmon.tamaraw20v2.csv`, `perfmon.tamaraw80.csv` und `perfmon.openworld.csv`, die im Quellverzeichnis `src/thesis/data/` der Arbeit abgelegt sind. Für `exp-tamaraw-20v2` liegt keine Disk-Telemetrie vor, da `perfmon.py` während dieses Laufs noch nicht aktiv war.

Tabelle B.5: `exp-tamaraw-20v2`: CPU- und RAM-Auslastung der virtuellen Maschine zum jeweiligen Zeitpunkt seit Experimentstart.

Zeit [h]	CPU [%]	RAM [%]
0.00	3.20	0.82
2.00	99.81	24.71
4.00	99.91	42.53
6.00	99.78	59.93
8.00	99.91	77.47
8.12	99.91	78.42

Tabelle B.6: `exp-tamaraw-80`: CPU-, RAM- und Disk-Auslastung der virtuellen Maschine zum jeweiligen Zeitpunkt seit Experimentstart.

Zeit [h]	CPU [%]	RAM [%]	Disk [%]
0.00	99.89	34.92	46.76
2.01	99.87	44.68	55.63
4.02	99.86	54.10	64.28
5.97	99.87	63.13	72.74
7.98	99.88	72.34	81.35
9.99	99.84	81.52	89.90
11.77	99.85	89.76	97.35

Tabelle B.7: `exp-tamaraw-openworld`: CPU-, RAM- und Disk-Auslastung der virtuellen Maschine zum jeweiligen Zeitpunkt seit Experimentstart. Der Lauf wird bei 16.10 Stunden durch `ENOSPC` abgebrochen, als die Disk die volle Belegung von 156 GB erreicht.

Zeit [h]	CPU [%]	RAM [%]	Disk [%]
0.00	5.68	1.81	24.73
2.02	99.86	24.61	34.16
3.97	99.86	34.75	43.41
5.99	99.86	45.34	52.93
8.01	99.82	55.62	62.41
10.03	99.84	65.84	71.79
11.98	99.81	75.72	80.74
14.00	99.84	85.87	90.08
16.02	99.82	95.94	99.41
16.10	99.80	96.41	99.79

C Begleitende Materialien

Dieser Anhang dokumentiert den Inhalt des dieser Arbeit beigelegten ZIP-Archivs. Das Archiv enthält den im Rahmen dieser Arbeit entwickelten Quellcode, die zugehörige Dokumentation sowie die aggregierte Resultattabelle `ExperimentLogs.csv`. Es ermöglicht, die in Kapitel 5 beschriebene Pipeline nachzuvollziehen und die in Kapitel 6 dargestellten Resultate auf einer vergleichbaren Maschine zu reproduzieren. Bewusst nicht enthalten sind grosse oder von Dritten stammende Bestandteile wie rohe Verkehrsmitsschnitte, Datensätze, trainierte Modelle und die WFLib-Codebasis, die sich aus dem enthaltenen Quellcode reproduzieren lassen und in Abschnitt C.2 einzeln aufgeführt sind. Die finale Bachelorarbeit als PDF und ihre LaTeX-Quellen werden separat abgegeben und sind nicht Teil dieses Code-Archivs.

C.1 Verzeichnisstruktur

Tabelle C.1 listet die wichtigsten Pfade des Archivs und deren Inhalt. Die Pfadangaben sind relativ zur Wurzel des Archivs und entsprechen der Struktur des Projekt-Repositoriums, sodass die relativen Pfade in den Skripten und in `README.md` unverändert gelten.

Tabelle C.1: Inhalt des beigelegten ZIP-Archivs.

Pfad	Inhalt
<code>README.md</code>	Einstiegspunkt mit Aufbau, Bedienung und Reproduktionsanleitung
<code>documentation.md</code>	Vertiefende Gesamtbeschreibung der Pipeline (Setup, Ablauf, Architektur)
<code>ExperimentLogs.csv</code>	Aggregierte Resultattabelle aller Experimente
<code>src/simulation/</code>	Skripte zur Steuerung der Shadow-Simulation (Abschnitt 5.1)
<code>src/simulation/conf/</code>	torrc-Vorlagen und Tamaraw-Bridge-Zertifikat
<code>src/simulation/generated/</code>	Index-ZIM-Datei erzeugte URL-Liste der Ziel-seiten
<code>src/ml/</code>	Skripte der ML-Pipeline (<code>pcap_to_npz.py</code> , <code>run_df.sh</code> , <code>correlation.py</code> , <code>pcap_overhead.py</code> , <code>plot_*.py</code> , <code>report.py</code>)

C.2 Nicht enthaltene Daten

Aus Platzgründen und zur Wahrung von Lizenz- und Urheberrechten werden die in Tabelle C.2 aufgeführten Bestandteile nicht im Archiv ausgeliefert. Für jeden Eintrag wird der Grund und der reproduzierende Schritt angegeben.

Tabelle C.2: Bewusst nicht ausgelieferte Bestandteile und ihre Reproduktion.

Bestandteil	Grund	Reproduktion
Rohe PCAPs der Monitor- und Exit-Hosts Experiment-Konfigurationen (exp-*/ mit shadow.config.yaml, schedule.json, sim.log)	Insgesamt rund 250 GB, sprengt die übliche Archivgrösse Werden bei jedem Lauf aus den enthaltenen Skripten neu erzeugt	<code>python3 shadowctl.py run <name></code> mit der jeweiligen Konfiguration (Abschnitt C.3) Entstehen automatisch bei <code>shadowctl.py run</code>
Datensätze (.npz), trainierte DF-Modelle und Ergebnis-JSONs	Aus den PCAPs ableitbar, keine Primärdaten	<code>pcap_to_npz.py</code> und <code>run_df.sh</code> (Abschnitt C.3)
Python-Virtualenvs (venv/)	Maschinen- und versionsspezifisch	Neu anlegen via <code>python3 -m venv venv</code> und Pakete aus <code>documentation.md</code>
WFLib-Codebasis (src/ml/wflib/)	Drittprojekt unter eigener Lizenz [33]	Klonen aus https://github.com/Xinhao-Deng/Website-Fingerprinting-Library
Tornettools-Eingabedaten (Konsensus, Server-Descriptors)	Drittprojekt, gross und versionsspezifisch	Bezug über https://github.com/shadow/tornettools und das dort dokumentierte CollecTor-Snapshot-Verfahren

C.3 Reproduktion eines Resultats

Zur Veranschaulichung wird hier exemplarisch die Reproduktion des Closed-World-Laufs `exp-baseline-80` skizziert, dessen Ergebnis in Tabelle C.3 dokumentiert ist.

1. Simulation starten und Pcaps erzeugen: aus `src/simulation/` den Befehl `python3 shadowctl.py run exp-baseline-80 --pages 80 --visits 150 --monitors 20 --padding off` absetzen und die Pcaps anschliessend mit `python3 shadowctl.py pull-results -name exp-baseline-80` herunterladen.
2. Pcaps in das WFLib-Format konvertieren: aus `src/ml/` den Befehl `python3 pcap_to_npz.py --schedule ../simulation/exp-baseline-80/shadow.config.schedule.json --shadow-data ../simulation/exp-baseline-80/results/shadow.data --out wflib/datasets/ExpBaseline80.npz` ausführen.
3. Daten splitten, trainieren und testen: aus `src/ml/` das Wrapper-Skript `bash run_df.sh ExpBaseline80` aufrufen, das die in `wflib/exp/` liegenden WFLib-Skripte mit den in dieser Arbeit verwendeten Hyperparametern startet.
4. Resultate ablesen: `cat wflib/logs/ExpBaseline80/DF/result.json` liefert die in Tabelle C.3 verglichenen Kennzahlen.

Da das Archiv aus Platzgründen weder vorkonvertierte Datensätze noch trainierte Modelle enthält, durchläuft die Reproduktion die vollständige Kette von der Simulation über die Konvertierung bis zum Testlauf. Die dafür benötigte WFlib-Codebasis und das Python-Environment werden zuvor gemäss `README.md` und `documentation.md` eingerichtet.

C.4 Tabellarische Gesamtsicht aller Hauptexperimente

Tabelle C.3 fasst die in Kapitel 6 ausgewerteten zwölf Hauptexperimente in einer einzigen Übersicht zusammen. Die Werte entsprechen direkt der Datei `ExperimentLogs.csv` im Hauptverzeichnis des begleitenden Repositoriums und sind auf die in dieser Arbeit ausgewerteten Spalten reduziert. Konstante Spalten wie Netzwerk-Skala, Seed, ZIM-Datei oder Trainings-Epochen werden weggelassen, da sie für alle Hauptexperimente identisch sind (Network Scale 0.010, Seed 1, ZIM `simple_en_maxi_2026-02`, 30 Trainings-Epochen, Visit-Interval 30 s, Visits pro Seite 150). Explorative Läufe (Smoketests, frühe Korrelations- und Tamaraw-Vorversuche) sind nicht enthalten, sind in der vollständigen CSV-Datei aber dokumentiert.

Tabelle C.3: Aggregierte Kennzahlen der zwölf Hauptexperimente. Quelle: `ExperimentLogs.csv`.

Experiment	Klassen	Padding	Wall-Clock	Samples	/Klasse	Fail %	Pkt/Sample	Acc. %	Prec. %	Rec. %	F1 %	Rand. %
exp-baseline-20	20	OFF	07:57	3 000	150	1.7	635	92.7	93.3	92.7	92.7	5.0
exp-padding-20	20	ON	08:16	3 000	150	1.5	638	83.3	84.2	83.3	82.8	5.0
exp-reduced-20	20	Reduced	08:18	3 000	150	1.9	611	86.7	88.1	86.7	86.3	5.0
exp-tamaraw-20v2	20	Tamaraw	08:19	3 000	150	1.6	5 000	39.3	50.7	39.3	38.6	5.0
exp-baseline-80	80	OFF	15:48	12 000	150	4.4	583	82.4	81.8	82.4	81.4	1.2
exp-padding-80	80	ON	16:01	12 000	150	4.5	601	78.6	78.5	78.6	76.9	1.2
exp-reduced-80	80	Reduced	15:39	12 000	150	4.1	591	77.5	76.7	77.5	76.2	1.2
exp-tamaraw-80	80	Tamaraw	16:47	12 000	150	2.9	5 000	31.6	39.4	31.6	29.2	1.2
exp-openworld	280	OFF	17:48	13 740	146	3.1	601	68.3	67.9	74.6	69.8	1.2
exp-openworld-on	280	ON	18:00	13 680	146	3.3	604	70.7	72.5	75.6	72.5	1.2
exp-openworld-reduced	280	Reduced	18:03	13 680	146	3.1	592	71.9	73.3	76.4	73.6	1.2
exp-tamaraw-openworld	280	Tamaraw	16:45	12 620	132	3.2	4 994	28.0	26.6	21.4	20.0	1.2

Die Spalten haben folgende Bedeutung. Klassen entspricht der Anzahl der vom Klassifikator unterschiedenen Seiten (Closed-World) beziehungsweise der Anzahl Quellseiten (Open-World: 280 Quellseiten, davon 80 überwachte Klassen plus eine Sammelklasse). Padding bezeichnet die im Tor-Daemon der Monitor-Hosts gesetzte Verteidigung (OFF, ON, Reduced) beziehungsweise das auf den Bridge-Hosts aktivierte Tamaraw. Wall-Clock ist die Laufzeit der Shadow-Simulation in Stunden:Minuten. Samples und /Klasse geben die nach Konvertierung mit `pcap_to_npz.py` verfügbaren Aufzeichnungen wieder. Fail % beziffert den Anteil der durch `wget2` abgebrochenen Seitenabrufe. Pkt/Sample ist die mittlere Anzahl Pakete pro Aufzeichnung über die ersten 5000 Werte des direction-Vektors. Acc., Prec., Rec. und F1 stammen aus dem WFLib-Testlauf des DF-Modells. Rand. ist die theoretische Zufallsbasis $1/K$ mit K effektiv vom Klassifikator unterschiedenen Klassen (Open-World: $81 = 80$ überwachte + 1 Sammelklasse, daher 1.2 %).

C.5 Verteilung der Sequenzlänge pro Besuch

Tabelle C.4 ergänzt die in Abschnitt 6.1 herangezogenen Sanity-Kontrollen um die empirische Verteilung der Sequenzlänge pro Besuch und dokumentiert, in welchem Umfang die in Abschnitt 3.11 festgelegte Obergrenze von 5000 Werten greift.

Tabelle C.4: Verteilung der Sequenzlänge pro Besuch nach der Konvertierung mit `pcap_to_npz.py`. Eigene Auswertung über alle Samples des jeweiligen Datensatzes.

Experiment	N	Median	p90	Max	Saturiert (≥ 5000)
exp-baseline-20	3 000	623	854	1 270	0.0 %
exp-padding-20	3 000	623	832	1 247	0.0 %
exp-reduced-20	3 000	596	827	1 224	0.0 %
exp-tamaraw-20v2	3 000	5 000	5 000	5 000	100.0 %
exp-baseline-80	12 000	581	733	2 303	0.0 %
exp-padding-80	12 000	601	734	2 253	0.0 %
exp-reduced-80	12 000	591	736	2 311	0.0 %
exp-tamaraw-80	12 000	5 000	5 000	5 000	100.0 %
exp-openworld	13 740	597	747	2 399	0.0 %
exp-openworld-on	13 680	599	732	2 270	0.0 %
exp-openworld-reduced	13 680	588	734	2 454	0.0 %
exp-tamaraw-openworld	12 620	5 000	5 000	5 000	99.8 %

D Ausgewählte Konfigurations- und Code-Auszüge

Dieser Anhang sammelt drei Auszüge aus den in Kapitel 5 beschriebenen Skripten und Konfigurationsdateien. Sie dienen als Beleg für die im Fliesstext zusammengefasste Funktionsweise und erlauben es, die Pipeline anhand der konkreten Dateiformate nachzuvollziehen.

D.1 Schedule-Datei

Die von `generate-wf-config.py` erzeugte Datei `schedule.json` dient als Ground Truth für die spätere Zuordnung von PCAP-Abschnitten zu Seitenaufrufen (vgl. Abschnitt 5.2.4). Pro Monitor enthält sie eine Liste der geplanten Besuche mit Startzeit, Zielport und URL.

```
"monitor0": [
  {
    "start_time": 1200,
    "port": 8000,
    "url": "http://129.114.108.192:8000/
          Eurovision_Song_Contest_2018"
  },
  {
    "start_time": 1230,
    "port": 8010,
    "url": "http://129.114.108.192:8010/Ionizing_radiation"
  }
]
```

Listing D.1: Auszug aus `schedule.json`: für jeden Monitor werden Startzeit, Zielport und URL der geplanten Besuche festgehalten.

D.2 URL-Liste

Die von `generate-urls.py` erzeugte Datei `urls.txt` legt fest, welche Wikipedia-Seiten den Monitor-Hosts zugewiesen werden (vgl. Abschnitt 5.3). Pro Zeile sind die IP des Webserver, der für die Seite verwendete Port und die vollständige URL angegeben.

```
129.114.108.192 8000 http://129.114.108.192:8000/
    Eurovision_Song_Contest_2018
129.114.108.192 8001 http://129.114.108.192:8001/Andhra_Pradesh
129.114.108.192 8002 http://129.114.108.192:8002/Rajon_Rondo
129.114.108.192 8003 http://129.114.108.192:8003/Hagerman,_Idaho
129.114.108.192 8004 http://129.114.108.192:8004/Echzell
```

Listing D.2: Beispielhafter Auszug aus `urls.txt`. Jede Zeile enthält IP, Port und URL einer ausgewählten Wikipedia-Seite.

D.3 NEWNYM-Skript

Das Skript `newnym.py` erzwingt vor jedem Seitenbesuch einen frischen Tor-Circuit (vgl. Abschnitt 5.4.3). Es öffnet einen Socket zum lokalen Tor-Control-Port und sendet das Signal `NEWNYM`.

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("127.0.0.1", 9051))
    s.sendall(b"AUTHENTICATE\r\n"); s.recv(1024)
    s.sendall(b"SIGNAL NEWNYM\r\n"); s.recv(1024)
```

Listing D.3: `newnym.py`: minimaler Tor-Control-Port-Client, der vor jedem Besuch einen frischen Circuit erzwingt.